8 BÖLÜM · PROBLEM → VERİ → MODEL → YAZILIM → GA → SONUÇLAR → GELECEK

Starship Kalkışında İleri Beslemeli, Kazanç Programlı PID Kontrolü

Kontrol kazançlarının Genetik Algoritma ile optimizasyonu · Ara ödev · Adım adım, basit anlatım · Melih Demir · Optimizasyon Dersi

İçindekiler

01 **Problem ve amaç**
Starship, misyon, roketin yapısı

02 **Veri**
Kaggle IFT-6/7/8, kalkışın ilk 150 s

03 **Temizleme ve referans**
Polinom, γ, max-Q, RMSE

04 **Sahte roket ve kontrol**
Model, PID + FF + kazanç programlama, senaryolar, J

05 **Yazılım**
Python dosyaları ve yol haritası

06 **Genetik Algoritma**
Turnuva, çaprazlama, mutasyon, nesil

07 **Deneyler ve sonuçlar**
Tablolar, şekiller, ablasyon

08 **Özet ve gelecek**
Genel özet, devam eden çalışmalar

01

Problem: Starship ve çalışmanın amacı

Roketin yapısı, görevi ve bu çalışmada neyi optimize ettiğimiz.

Amaç ve misyon

Çalışmanın amacı

- Derste teorik olarak öğrendiğimiz optimizasyon yöntemlerini **gerçek hayata uygulamak**.
- Starship roketinin kalkışta hızını kontrol eden **ileri beslemeli, iki fazlı (kazanç programlı) PID** kontrolcünün **6 kazancını** elle tahmin etmek yerine **Genetik Algoritma (GA)** ile otomatik bulmak.
- Bulunan ayarı **elle ayarlanmış PID** ve **rastgele arama** ile karşılaştırıp ne kazandırdığını ölçmek.

Starship'in misyonu

- **Tamamen yeniden kullanılabilir** dev bir roket yapmak.
- Booster da Ship de geri dönüp tekrar uçacak; böylece uzaya gitmek **çok daha ucuz** olacak.
- Hedefler: büyük Starlink uydularını taşımak, Ay'a iniş (NASA Artemis programı) ve uzun vadede **Mars**.

Kısaca: Gerçek roket verisi + ileri besleme + iki fazlı PID + Genetik Algoritma → en iyi PID ayarı

Gerçek uçuş: kalkış ve iniş





Neye bakmalı?

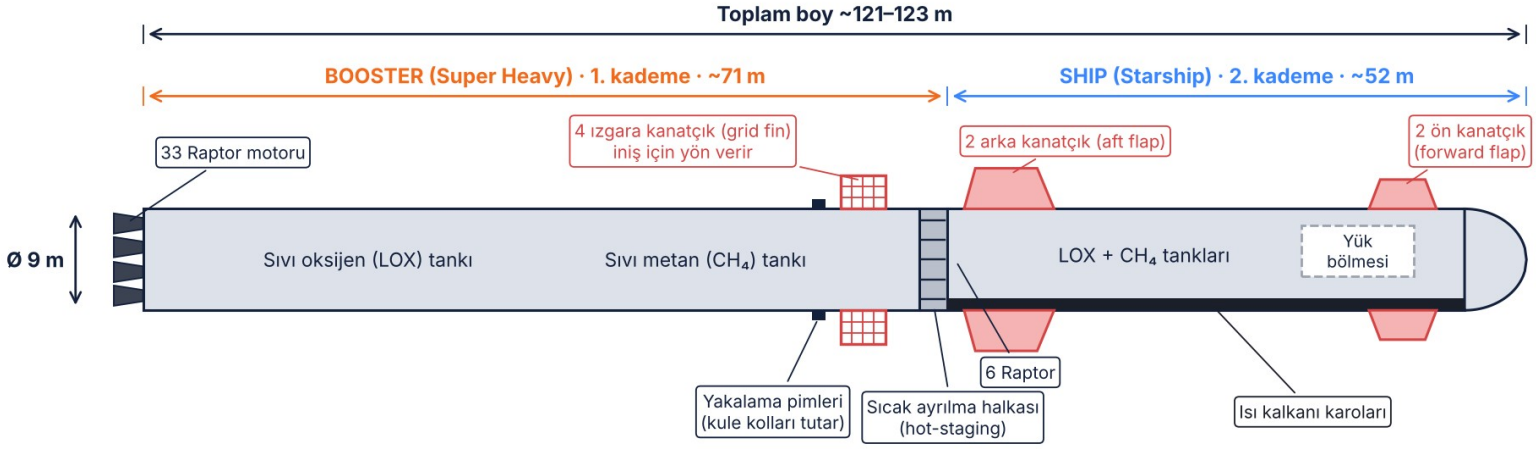
- Ekrandaki T+ saatini takip et
- Kalkış: T+00:00 – 02:40 (bizim veri: ilk 150 s)
- Ayrılma: ~T+02:41
- İniş yanması: T+06:35 – 07:00
- Yakalama: ~T+07:00

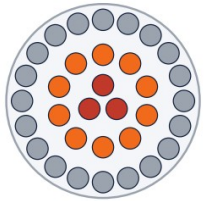
▶️ Video oynamazsa: YouTube'da aç

Ekranın altındaki hız ve irtifa sayıları, veri setinin OCR ile okunduğu yer.

Roketin yapısı: Booster ve Ship

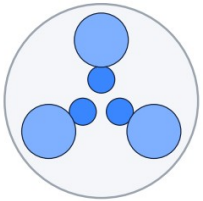
Roket iki parçadan oluşuyor: **üst kısım Ship (Starship)**, **alt kısım Booster (Super Heavy)**. Kalkışta ikisi birbirine bağlı, tek parça uçuyor.





Booster altı: 33 motor

- Merkez 3 + iç halka 10 = 13 yönlenebilir (gimbal)
- Dış halka 20 = sabit, sadece kalkışta
- İnişte önce 13, sonra merkez 3 motor



Ship altı: 6 motor

- Merkez 3: deniz seviyesi, yönlenebilir
- Dış 3: vakum motoru (RVac), büyük nozul, uzayda verimli

Şema ölçekli değildir; ölçüler yaklaşık, kamuya açık değerlerdir.

Ölçüler, ağırlık ve itki

	Booster (Super Heavy)	Ship (Starship)
Boy	~71 m	~52 m
Çap	9 m	9 m
Motor	33 Raptor (merkez 3 + iç 10 + dış 20)	6 Raptor (3 deniz seviyesi + 3 vakum)
Yön verme	13 motor yönlenebilir (gimbal) + 4 ızgara kanatçık	3 merkez motor gimbal + 4 kanatçık (2 ön, 2 arka)
Yakıt (sıvı metan + sıvı oksijen)	~3.400 ton	~1.500 ton
Kuru kütle (yakıtsız)	~200–275 ton (tahmini)	~100–130 ton (tahmini)
Motor başına itki	~2,26 MN (~230 ton-kuvvet)	~2,3–2,5 MN
Toplam itki	33 × 2,26 ≈ 74,6 MN (~7.600 ton-kuvvet)	~14 MN (uzayda)

Kalkış kütlesi ~5.000 ton → ağırlık ≈ 49 MN. İtki 74,6 MN → itki/ağırlık ≈ 1,5: roket kendi ağırlığının 1,5 katı kuvvetle itiliyor, bu yüzden yerden kalkabiliyor.

Değerler yaklaşık; SpaceX kesin değerleri yayımlamıyor. Modelde kullanılanlar: src/config.py (M0 = 5000 t, 33 motor, MDOT = 705 kg/s, Isp = 327 s).

Görev adımları: Booster ve Ship ayrı ayrı

Booster

1) 00:00–02:35 Kalkış: booster motorları tam güçte çalışıyor (bizim veri: ilk 150 s)

2) ~02:35: motorların çoğu kapanıyor

3) ~02:41 Ayrılma: Ship motorlarını yakıyor, booster'dan ayrılıyor

4) ~02:45–03:30 Geri dönüş yanması (boostback): motorları tekrar açıp kuleye doğru yön değiştiriyor

5) ~03:30–06:30: motorlar kapalı; ~04:00'te ~87 km'de tepe, sonra serbest düşüş

6) ~06:30–07:00 İniş yanması (13 → 3 motor) ve kuleye kenetlenme

Ship

1) 00:00–02:41: booster'a bağlı, onunla birlikte çıkıyor (motorları kapalı)

2) ~02:41 Ayrılma: kendi 6 motorunu yakıp booster'dan ayrılıyor (hot staging)

3) ~02:43–08:30: yanmaya devam; ~150–190 km yüksekliğe, ~26.000 km/saat hıza çıkıyor

4) ~08:30: motorlar kapanıyor; Dünya'nın etrafında neredeyse tam tur süzülüyor, uzayda testler yapıyor

5) ~45–60. dk: atmosfere giriş, ısı kalkanı ~1400 °C

6) ~65. dk: Hint Okyanusu'na kontrollü iniş

Bizim çalışma: Booster'ın 1. adımı (kalkış, ilk 150 s). İrem: Booster'ın 6. adımı (iniş yanması).

02

Veriyi aldık #kaggle csv dosyaları

Üç uçuşun Booster sütunlarından sadece **hız** ve **irtifa** bilgisini, kalkışın ilk **150 saniyesi** için aldık. Excel'de gri boyalı satırlar tam olarak bunlar.

Kalkış mı, iniş mi? İrtifaya bak

Sütun	Kalkış (00:00–02:30): 150 saniye	İniş (06:30–07:00): 30 saniye
İrtifa	Artıyor: 0 → 56 km	Azalıyor: 3 → 0 km
Hız	Artıyor: 0 → 4446 km/h	Azalıyor: 1307 → 2 km/h
İvme	Pozitif: hızlanıyor	Negatif: yavaşlıyor
Yakıt	Hızla düşüyor: %94 → %10	Neredeyse bitmiş: %1–3

Kısaca: Roket yukarı çıkarken irtifası artar (kalkış), aşağı inerken irtifası sıfıra düşer (iniş); satırları buna bakarak ayırdık.

Veri nereden geliyor?

1 **SpaceX canlı yayını**

Ekranın altında hız ve irtifa yazıyor

→

2 **OCR**

Bilgisayar ekrandaki sayıları saniye saniye okuyor

→

3 **CSV tablosu**

Her saniye bir satır: zaman, hız, irtifa, yakıt...

IFT-6

- Gerçek bir test uçuşu
- Kasım 2024
- Sütunlar aynı, sayılar farklı

IFT-7

- Gerçek bir test uçuşu
- Ocak 2025
- Sütunlar aynı, sayılar farklı

IFT-8

- Gerçek bir test uçuşu
- Mart 2025
- Sütunlar aynı, sayılar farklı

* *Veri kaynağı: arya2209, "Starship-Launches-Dataset-ift1-ift8", Kaggle.* https://www.kaggle.com/datasets/arya2209/starship-launches-dataset-ift1-ift8

03 Veriyi temizleyip hedef çıkardık #src/referans.py

- Aynı saniye iki kez yazılmışsa birini attık.
- Hızı km/h'den m/s'ye çevirdik.
- OCR'dan gelen gürültüyü **7. derece polinomla yumuşattık**: noktaların arasından geçen düzgün bir eğri çizdik.
- İrtifa sadece tam km olarak verildiği için onu da **6. derece polinomla** yumuşattık. Buradan roketin **ne kadar eğik uçtuğunu (uçuş yolu açısı γ)** hesapladık.
- **Max-Q'yu hesapladık:** Max-Q roketin üzerindeki hava basıncının en yüksek olduğu andır. Değeri ~22 kPa olarak bulduk. Bu değere yaklaşırken motorların gazı kısılmalı; yoksa roket hasar alabilir.

Neden temizliyoruz?

- **Gürültü:** OCR okumaları titriyor
- **Aykırı değer:** arada imkânsız okumalar var
- **Tekrar:** aynı an birden fazla kaydedilmiş, siliyoruz
- **Sonuç:** temizlemezsek ivme zıplar, PID yanlış tepki verir

Sonuç: Her uçuş için "şu saniyede şu hızda olmalısın" diyen bir **hedef hız eğrisi** elde ettik.

OCR'ın iki tür hatası

Gürültü (titreme)

- Gerçek değerin etrafında küçük, rastgele sapmalar
- Örneğin hız artışı her saniye +22, +22, +11, +12, +23 şeklinde okunabiliyor. Bu durumda orada yanlış değerler oluşuyor.
- Roket aslında düzgün hızlanıyor; okuma zamanlaması kayıyor. Yani roket hiç yavaşlamadı; sadece iki fotoğraf arası süre 1 saniye değil yarım saniye oldu. Program her seferinde 1 saniye geçti sandığı için artış yarıya düşmüş görünüyor.
- Çözüm: Polinomla yumuşatma

Aykırı değer (outlier)

- Tek seferlik, büyük, imkânsız bir okuma
- Örnek: 06:46'da hız "0" okunmuş. Roket o an ~170 km/h ile iniyordu
- Çözüm: Komşu değerlerden doldurma (interpolasyon)

Neden önemli? Gürültülü veriden ivme hesaplanırsa ivme zıplar, PID boşuna gaz kolunu oynatır.

Polinomla yumuşatma

Ne yaptık?

- Polinom = kıvrılabilen düzgün bir eğri formülü
- Derece 1 düz çizgi, 2 parabol, 7 birkaç yumuşak kıvrım
- Bilgisayar eğriyi noktalara en yakın olacak şekilde çiziyor (en küçük kareler)
- Kodda tek satır: np.polyfit(t, v, 7)

Saniye	Ham (m/s)	Yumuşak (m/s)	Artış
42	200,8	198,1	
43	206,9	203,6	+5,5
44	210,1	209,0	+5,4
45	213,3	214,5	+5,5
46	219,7	220,0	+5,5

Yumuşak eğride roket her saniye düzgünce ~5,5 m/s hızlanıyor. Ham veriyle fark 2–4 m/s.

Özetle: "OCR gürültüsünü en küçük kareler ile 7. derece polinom uydurarak yumuşattık."

7. derece polinom: adım adım

1. **150 titrek nokta**
   Her saniye bir hız değeri var; OCR yüzünden zıplıyor.

2. **Bir eğri formülü seç**
   $v(t) = a_0 + a_1t + a_2t^2 + \ldots + a_7t^7 \rightarrow$ 8 bilinmeyen sayı ($a_0 \ldots a_7$).

3. **Neden 7. derece?**
   1 = düz çizgi (çok kaba), 2 = tek kıvrım. 7 max-Q'daki yavaşlamayı yakalar; 50 olsa gürültüyü de kopyalar.

4. **Mesafeleri ölç**
   Her noktanın eğriye uzaklığını al, karesini al, hepsini topla.

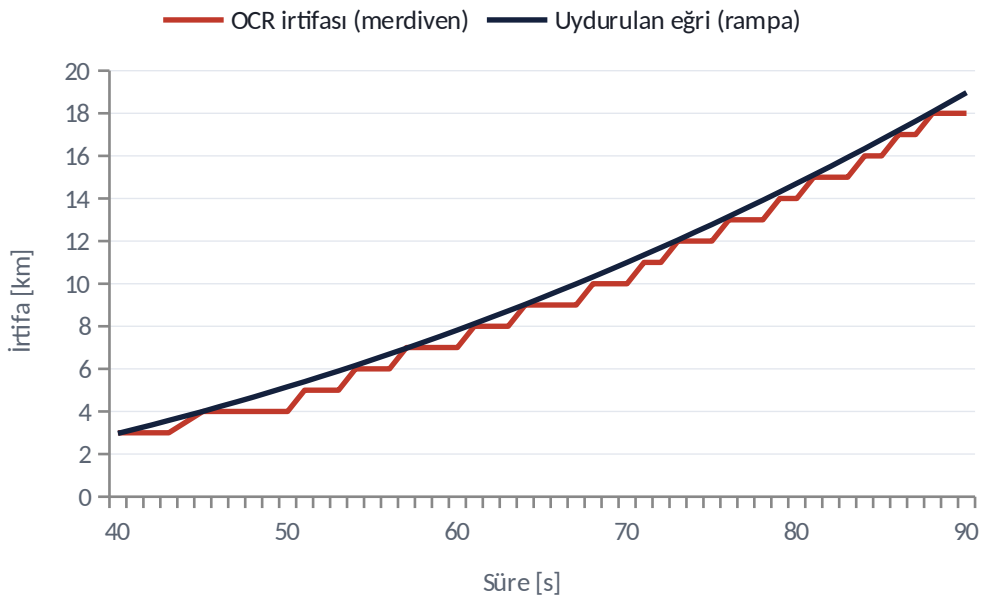
5. **En küçüğü bul**
   Bu toplamı en küçük yapan $a_0 \ldots a_7$'yi bilgisayar bulur: np.polyfit(t, v, 7). Adı: en küçük kareler.

6. **Eğriyi kullan**
   Artık ham noktalar yerine düzgün eğri hedef hız oluyor.

7. **Eğimden ivme**
   Eğrinin eğimi = hedef ivme. PID bunu ileri besleme olarak kullanıyor; zıplamadığı için gaz kolu boşuna oynamıyor.

Sonuç: eğri ile ham veri arasındaki ortalama fark (RMSE) sadece 2,3–3,6 m/s; hız ise ~1200 m/s.

Merdiven değil rampa: irtifa



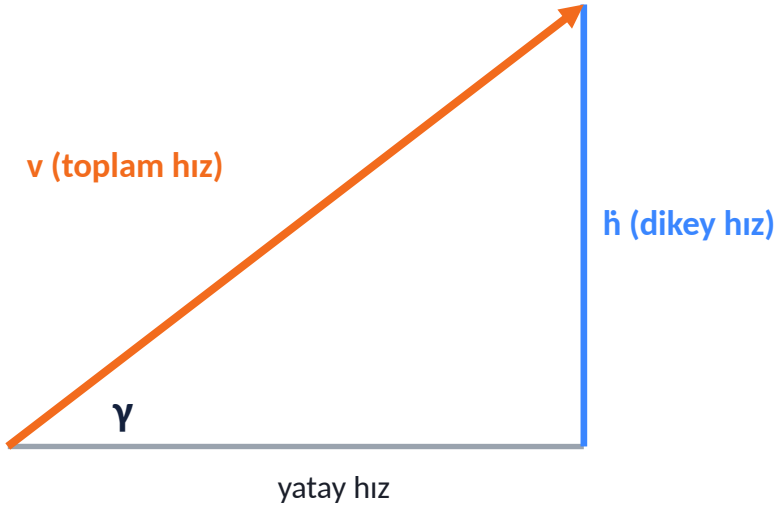
Saniye	OCR	Basamak	Rampa
59	7 km	0 m/s ❌	287 m/s ✔️
60	7 km	0 m/s ❌	292 m/s ✔️
61	8 km	1000 m/s ❌	297 m/s ✔️

Dikey hız = irtifanın saniyede ne kadar arttığı = rampanın eğimi.

Merdivenin eğimi ölçülemez, rampanın eğimi ölçülür.

Uçuş yolu açısı γ: roket ne kadar yatık?

- **Roket sadece yukarı çıkmıyor:** yörüngeye girmek için yana yatıp yatay hız kazanması gerekiyor.
- **γ'ya bu yüzden ihtiyacımız var:** yerçekiminin roketi ne kadar yavaşlattığı (g · sin γ) bu açıya bağlı. Bu açıyı bilmeden sahte roketin hızını ve yüksekliğini doğru hesaplayamayız.



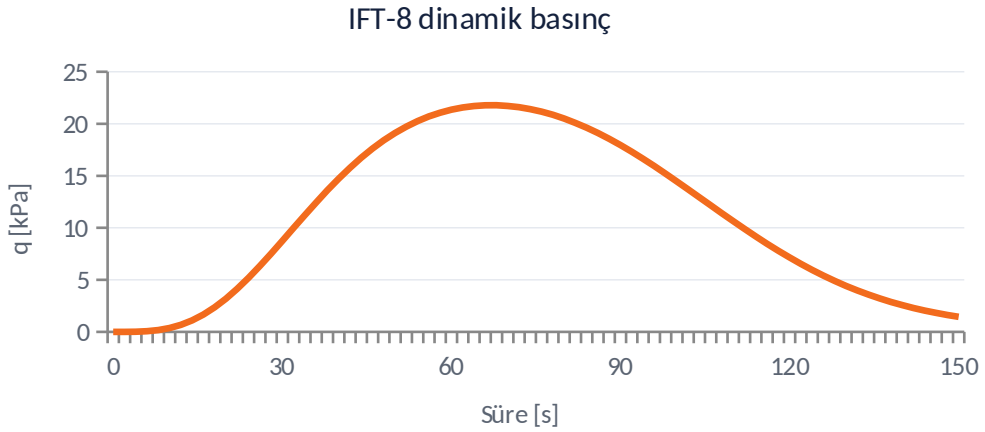
Süre	V	ḣ	Γ
10 s	25 m/s	25 m/s	90° dik
60 s	299 m/s	292 m/s	77°
100 s	629 m/s	508 m/s	54°
150 s	1213 m/s	692 m/s	35° yatık

Kontrol: bu açıyla uçan model 150. saniyede 54,7 km'de; veri 54 km diyor.

sin γ = dikey hız / toplam hız. Modelde yerçekiminin roketi yavaşlatan kısmı: g · sin γ.

Max-Q: roketin en çok zorlandığı an

Kalkış sırasında roketin üzerine binen hava basıncının **en yüksek** olduğu an. Yaklaşık **60–67. saniyede ~22 kPa** ile tepe yapıyor; gövde en çok bu anda zorlanıyor. Gerçek roket max-Q civarında gazı biraz kısar. Biz de PID'e bu basıncı **%5'ten fazla aşmama** kuralı koyduk. 22 kPa'yı 67. saniyedeki değerleri formüle koyarak bulduk (sonraki sayfa).



$q = \frac{1}{2} \cdot \rho \cdot v^2$

- v (hız) artıyor → q büyür
- ρ (hava) inceliyor → q küçülür
- Ortada tepe: ~22 kPa, 67. saniye = max-Q
- Gövde en çok burada zorlanır
- Sınır: 1,05 × max-Q ≈ 23 kPa

Sonuç: 200 test senaryosunun hiçbirinde sınır aşılmadı.

22 kPa nasıl bulundu? Adım adım

1 Formül

$q = \frac{1}{2} \cdot \rho \cdot v^2$ → basınç = ½ × hava yoğunluğu × hız²

2 67. saniyedeki hız

Yumuşatılmış hız eğrisinden: **v = 343 m/s** (~1235 km/h)

3 67. saniyedeki hava

Yükseklik **h ≈ 10,2 km**. Hava yukarıda incelir:
ρ = 1,225 × e^(−10177 / 8500) = **0,370 kg/m³**

4 Formüle koy

$q = \frac{1}{2} \times 0{,}370 \times 343^2 = \frac{1}{2} \times 0{,}370 \times 117.649$
= **21.765 Pa ≈ 21,8 kPa ≈ 22 kPa**

Saniye	Hız v	Yükseklik h	Hava ρ	q
20	75 m/s	0,6 km	1,145	3,2 kPa
40	187 m/s	3,2 km	0,841	14,7 kPa
60	299 m/s	8,0 km	0,477	21,4 kPa
67	**343 m/s**	**10,2 km**	**0,370**	**21,8 kPa**
80	439 m/s	14,9 km	0,212	20,5 kPa
100	629 m/s	24,0 km	0,073	14,5 kPa
150	1213 m/s	54,7 km	0,002	1,4 kPa

IFT-8 verisi. Hız artıyor ama hava çok hızlı inceliyor; çarpım 67. saniyede en büyük oluyor.

Neden 67. saniye? Her saniye için q'yu hesapladık, en büyüğü seçtik. IFT-6: 60 s / 21,8 kPa · IFT-7: 67 s / 22,3 kPa · IFT-8: 67 s / 21,8 kPa. Kodda: referans.py → q = 0.5*rho*v**2; q_max = q.max()

referans.py çıktısı: hedef yol hazır mı?

```
IFT-6: v(150)=1394 m/s, h_ref(150)=59.1 km (OCR 59 km), max-Q=21.8 kPa @ t=60 s, hız RMSE=2.31 m/s
IFT-7: v(150)=1172 m/s, h_ref(150)=56.1 km (OCR 56 km), max-Q=22.3 kPa @ t=67 s, hız RMSE=3.56 m/s
IFT-8: v(150)=1213 m/s, h_ref(150)=54.7 km (OCR 54 km), max-Q=21.8 kPa @ t=67 s, hız RMSE=2.99 m/s
```

Çıktı (IFT-8)	Ne demek?
v(150) = 1213 m/s	150. saniyedeki hedef hız (~4370 km/h)
h_ref(150) = 54,7 km	Bizim hesapladığımız yükseklik (hız × sin γ toplandı)
OCR 54 km	Ekranda yazan yükseklik (karşılaştırma için)
max-Q = 21,8 kPa @ 67 s	En yüksek hava basıncı ve zamanı
hız RMSE = 2,99 m/s	Polinom eğrisi ile ham veri arasındaki ortalama fark

1 **γ doğru hesaplanmış**

Hesaplanan yükseklik (54,7 km) ekrandakiyle (54 km) tutuyor. Ekran tam km'ye aşağı yuvarlıyor, yani gerçek değer 54–55 km arası.

2 **Yumuşatma iyi**

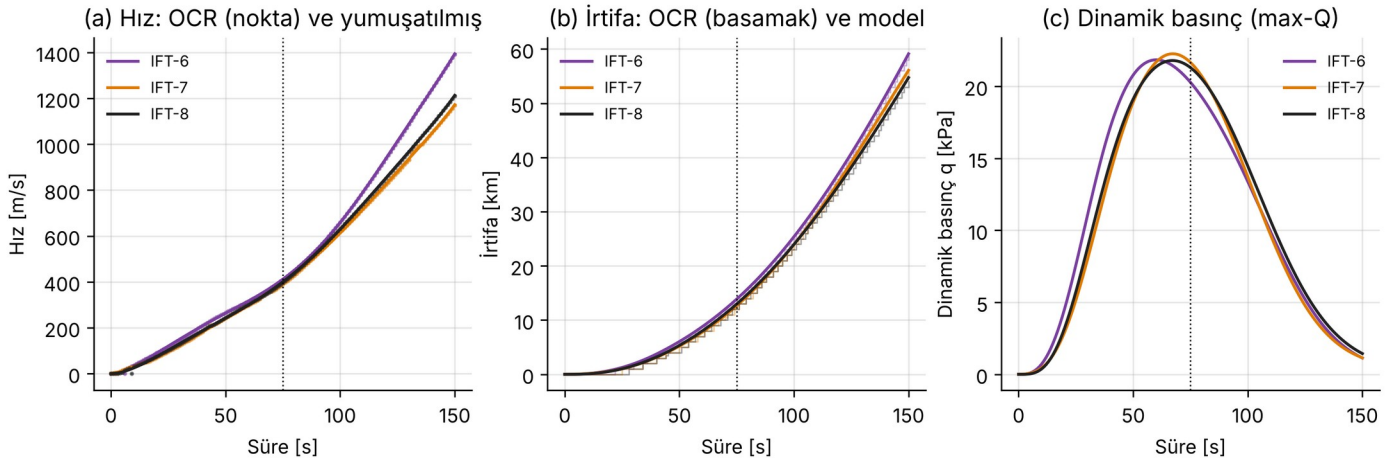
1200 m/s hızda 3 m/s fark = %0,25. Eğri veriyi takip ediyor, gürültüyü kopyalamıyor.

3 **Max-Q sınırı belli**

Model bu değeri kullanıyor: sınır = 1,05 × max-Q ≈ 23 kPa.

Üç satır = üç farklı gerçek uçuş = üç farklı "yol". IFT-6 daha hızlı çıkmış (1394 m/s). PID bu üç yolda da sınanıyor.

referans.py çıktısının grafiği



Grafik nerede?

Dosya:
sonuclar/sekiller/sekil1_kalkis_veri.png
Çizen kod:
src/grafikler.py
Bildiride: Şekil 1

(a) Hız

Noktalar ham OCR, çizgi 7. derece polinom. Çizgi noktaların üstünden geçiyor → **RMSE 2–4 m/s**. Sağ uç = v(150).

(b) İrtifa

Basamaklar ekrandaki tam km, çizgi hız × sin γ ile hesaplanan yükseklik → **h_ref ≈ OCR** (54,7 ≈ 54 km).

(c) Max-Q

$q = \frac{1}{2}\rho v^2$ her saniye. Tepeler **~22 kPa**, 60–67. s. Kesikli çizgi: 75 s, PID faz değişimi.

Neden referans.py çizmiyor? Bu dosya model.py ve optimizasyon her çalıştığında içe aktarılıyor; işi sadece hedef yolu hesaplamak. Çizim içine konsaydı her optimizasyon koşusu yavaşlardı. Bu yüzden iş bölümü yaptık: referans.py = hesap, grafikler.py = bütün şekiller (aynı stil, 300 dpi, bildiri için).

RMSE ne demek, neyle karşılaştırılır?

RMSE nedir?

- Eğrinin gerçek noktalardan **ortalama ne kadar uzak** olduğu.
- **4 adım:** farkı al → karesini al → ortalamasını al → karekökünü al.

Nokta	Fark	Karesi
1	+1	1
2	−3	9
3	+2	4

(1 + 9 + 4) / 3 = 4,67 → √ → **RMSE ≈ 2,16**

Kare alınır ki + ve − farklar birbirini götürmesin.

1) Hızın büyüklüğüyle karşılaştır

Uçuş	RMSE	Hız (150 s)	Oran
IFT-6	2,31	1394	%0,17
IFT-7	3,56	1172	%0,30
IFT-8	2,99	1213	%0,25

Dikkat: %17 değil, %0,17!

2,31 ÷ 1394 = 0,00166

× 100 → **%0,17** (yüzde)

× 1000 → **‰1,7** (binde ≈ 2)

Para örneği: 1394 TL'nin %17'si 237 TL olurdu; %0,17'si sadece **2,37 TL**. Bizim hata bu kadar küçük.

2) Diğer derecelerle karşılaştır

Polinom derecesi	RMSE (IFT-8)
1 (düz çizgi)	60,7
3	8,3
5	5,1
7 (bizim seçim)	**3,0**
20	2,0
40	1,75

7'ye kadar hızla düşüyor, sonra neredeyse hiç: **dirsek noktası**. Daha yüksek derece gürültüyü kopyalar.

3) OCR hatasıyla karşılaştır

Roket 5–15 m/s² hızlanıyor; ½ s okuma kayması ≈ **2,5–7 m/s** hata. RMSE (2–4) bu aralıkta.

Sonuç: RMSE hızın %0,2–0,3'ü (binde 2–3); 7. derece dirsek noktası ve OCR'ın beklenen hatasıyla uyumlu → yumuşatma iyi.

04

Bilgisayarda sahte bir roket yaptık #src/model.py

Gerçek roketi deneyemeyiz; fizik kurallarına uyan **sahte bir roketi bilgisayarda** canlandırdık. Bu bölümde 4 adım var:

1) **Sahte roket:** itki, yerçekimi, hava direnci, kütle ve motor gecikmesi
2) **PID'i kurduk:** şoför gaz kolunu nasıl ayarlıyor?
3) **Zor koşullar ekledik:** senaryolar
4) **Puanlama sistemi (J) kurduk:** hangi şoför daha iyi?

Bilgisayardaki sahte roket #src/model.py

150 saniyelik uçuş, **0,1 saniyelik 1500 adımda** hesaplanıyor. Her adımda roketin hızı, yüksekliği ve kütlesi güncelleniyor.

İtki (motor)

33 Raptor (ortada 3, iç halkada 10, dış halkada 20). Motor başına ~2,26 MN. Yükseldikçe biraz daha güçlü itiyor.

Yerçekimi

Roket yatık uçtuğu için yerçekiminin sadece g · sin γ kadarı hızı azaltıyor.

Hava direnci

$D = ½ \cdot \rho \cdot Cd \cdot A \cdot v^2$. Hızla büyür, hava inceldikçe küçülür.

Kütle

Kalkışta 5000 ton. Yakıt yandıkça hafifliyor, aynı itkiyle daha hızlı ivmeleniyor.

Döngünün kalbi (model.py, sadeleştirilmiş)

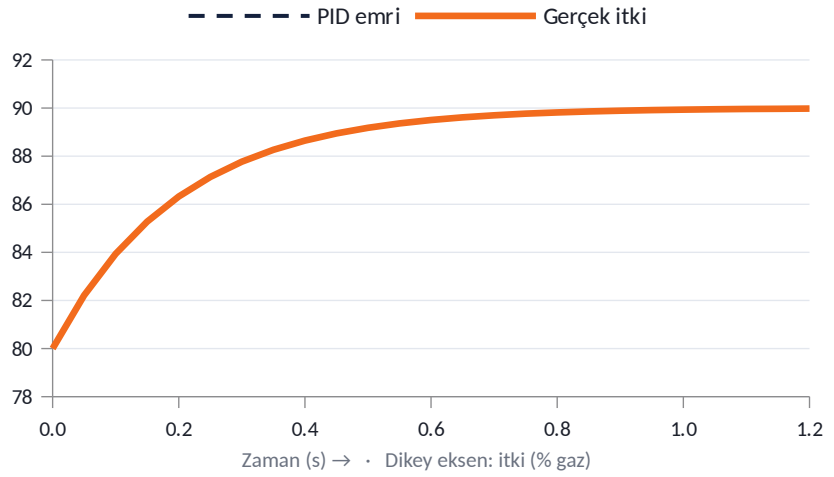
```
for k in range(1500):          # her 0,1 s
  T_motor = MDOT*isp*G0        # motor başına itki
  T_cmd = u*33*T_motor         # PID'in istediği
  T += (T_cmd-T)*(DT/TAU)      # motor gecikmesi
  D = 0.5*rho*cdA*v**2         # hava direnci
  a = (T-D)/m - G0*sg          # Newton: F = m·a
  v = v + a*DT                 # yeni hız
  h = h + v*sg*DT              # yeni yükseklik
  m = m - T/(isp*G0)*DT        # yakıt yandı
```

Özet: Newton'un 2. yasası → ivme = (itki − hava direnci) / kütle − yerçekimi. Her 0,1 s'de bu hesap tekrarlanıyor.

Motor gecikmesi: τ = 0,2 saniye

Ne demek?

- PID motora her 0,1 saniyede emir veriyor. Motor bu emri **anında** yerine getirmiyor.
- "Gazı %80'den %90'a çıkar" denince itki birden %90 olmuyor, **yavaş yavaş** %90'a yaklaşıyor.
- **Musluk gibi:** açınca su hemen tam akmaz, bir an sonra gürleşir. Roket motorunda yakıt pompalarının hızlanması ve yakıtın yanma odasına ulaşması zaman alıyor.
- **0,2 s** = farkın yaklaşık **üçte ikisinin** kapandığı süre. Mühendislikte buna **zaman sabiti (τ, tau)** denir.
- 0,2 s yayımlanmış bir değer değil; **bizim makul varsayımımız**.

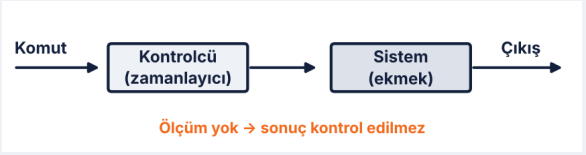
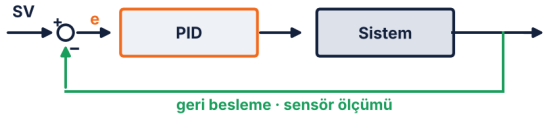
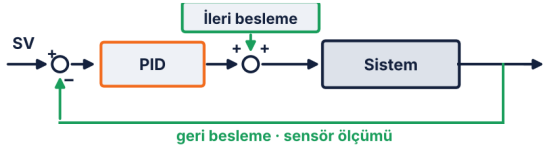


Süre	0 s	0,2 s	0,4 s	0,6 s	~1 s
İtki	%80	%86	%88,6	%89,5	%90

```
T += (T_cmd - T) * (DT / TAU_ENGINE)    # 0,1/0,2 = 0,5 → her adımda kalan farkın yarısı kapanır
```

Mühendislikte kontrol sistemleri

Ana ayrım: sistem **kendi sonucunu ölçüyor mu?** Ölçmüyorsa açık çevrim, ölçüp düzeltiyorsa kapalı çevrim (geri besleme).

Tür	Ne yapar?	Günlük örnek	Diyagram
Açık çevrim (open-loop)	Sonucu ölçmez, sadece komut verir.	Ekmek kızartma makinesi: 3 dk ısıtır, ekmek yandı mı bakmaz.	
Kapalı çevrim (closed-loop, feedback)	Sonucu ölçer, hatayı düzeltir. **PID burada.**	Klima: odayı ölçer, soğuksa ısıtır.	
İleri besleme (feedforward)	Bozucu etkiyi önceden bilip baştan karşılar.	Yokuşu görüp önceden gaza basmak.	
İleri + geri besleme	Büyük kısmı ileri besleme yapar, kalan hatayı PID düzeltir.	**Bizim roket modeli**	

Diğer kontrol yapıları ve bizim roket

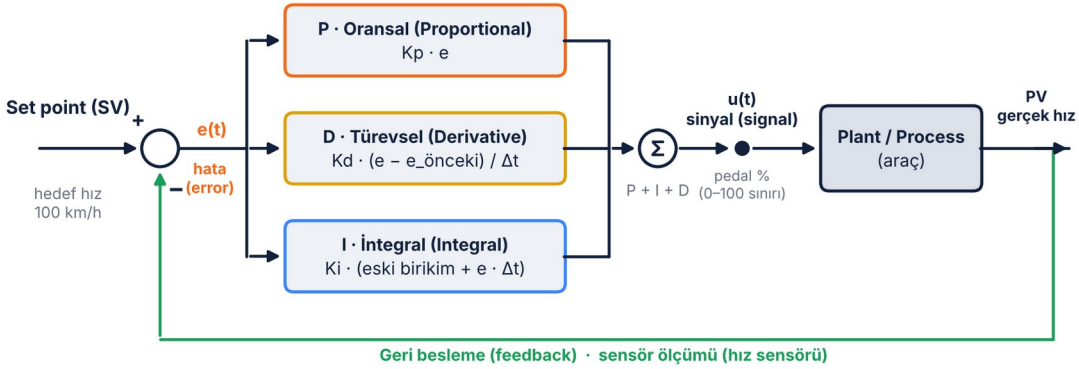
Yapı	Ne demek?	Günlük örnek	Bizde var mı?
Kazanç programlama (gain scheduling)	Duruma göre farklı PID ayarları kullanmak	Dolu / boş kamyon için farklı ayar	✅ Faz-1 (0–75 s), Faz-2 (75–150 s)
Kaskat (cascade) kontrol	İç içe iki döngü: dıştaki içtekine hedef üretir	Fırın: dış döngü sıcaklık, iç döngü gaz akışı	❌ (gerçek roketlerde var: konum → hız)
Model öngörülü kontrol (MPC)	Modelle geleceği tahmin edip en iyi hamleyi seçer	Otonom araç şerit takibi	❌ (daha gelişmiş yöntem)
Aç-kapa (on/off) kontrol	Sadece iki durum: açık ya da kapalı	Buzdolabı, basit termostat	❌

Bizim roket çalışmamızda üç şey birlikte kullanılıyor:

1. **Geri besleme (PID)**
   Ölçülen hatayı düzeltiyor
2. **İleri besleme**
   Bilinen ivme, yerçekimi ve hava direncini baştan veriyor
3. **Kazanç programlama**
   İki faz, iki ayrı PID ayarı

Sonraki sayfalar: önce her birinin basit örneği (cruise control → yokuş → kamyon), sonra roketimizde üçü birlikte.

PID kontrol: hız sabitleyici (cruise control) örneği



Δt = döngü süresi: her döngüde sinyal 0,5 s uygulanır (zaman 0 → 0,5 → 1 → 1,5 … ilerler, Δt hep 0,5 kalır).

Döngü 1 (Kp=2, Ki=0,1, Kd=0,2)	Hesap	Sonuç
Hata e	100 − 0	100
P / D / I	2×100 · önceki yok · sınırda → 0	200 / 0 / 0
u(t) → pedal → 0,5 s sonra hız	200 > 100 → %100	5 km/h

P · Oransal (Proportional)

P = Kp · e

Şu anki hataya bakar. Hata büyükse sert, küçükse yumuşak.

D · Türevsel (Derivative)

D = Kd · (e − e_önceki) / Δt

Hatanın ne kadar hızlı değiştiğine bakar; aşmayı önler. İlk döngüde önceki hata yok → D = 0.

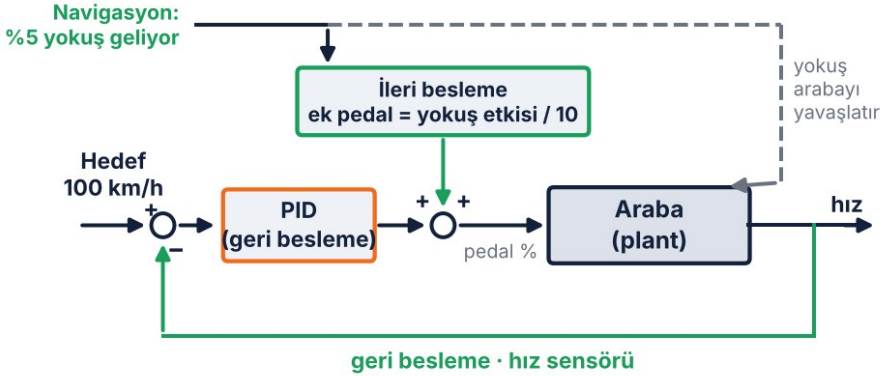
I · İntegral (Integral)

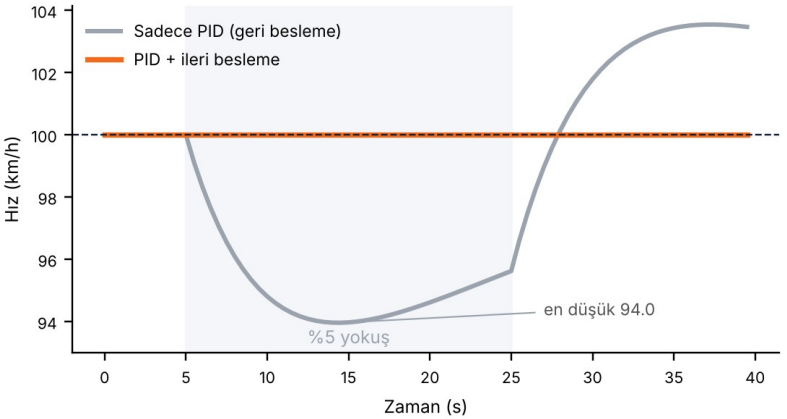
I_aday = Ki · (eski birikim + yeni birikim)
yeni birikim = e · Δt

Geçmişte biriken hatayı toplar; kalıcı farkı kapatır. Pedal sınırdaysa birikim büyütülmez (anti-windup).

U(t) = P + I + D → pedal (sinyal) → araç (plant) → sensör hızı ölçer → hata yeniden hesaplanır. Bu döngü her 0,5 s tekrarlanır.

İleri besleme örneği: yokuşu önceden görmek





- **Navigasyon %5 yokuşu biliyor.** Yokuş arabayı 0,49 m/s² ≈ 1,77 km/h her saniye yavaşlatır → ileri besleme baştan **1,77 / 10 ≈ %18 ek pedal** verir.
- **Sadece PID (gri):** yokuşta hız ~94 km/h'ye düşer; yokuş bitince birikmiş I yüzünden ~103,5'e fırlar.
- **PID + ileri besleme (turuncu):** yokuş tam biliniyorsa hız 100'de kalır. Tahmin biraz yanlışsa kalan farkı PID düzeltir.

Roketimizde ileri besleme: hedef ivme + yerçekimi (g·sinγ) + hava direnci önceden hesaplanıp PID'e eklenir.

%17,7 ek pedal nereden geliyor?

1 Yokuşun yavaşlatması: 1,77

- Yokuşta yerçekiminin **bir kısmı arabayı geriye çeker** (bisikletle yokuşta pedalı bırakınca yavaşlamak gibi).
- **%5 yokuş** = 100 m ilerlerken 5 m yükselmek → yerçekiminin ~%5'i geriye çeker.
- 9,81 × 0,05 = **0,49 m/s²** → × 3,6 = **1,77 km/h her saniye**
- Motor ekstra bir şey yapmazsa araba her saniye 1,77 km/h yavaşlar.

2 Ek pedal: %17,7

- **Tam gaz ivmesi = 10:** %100 pedalda araba her saniye 10 km/h hızlanır (Excel B13, varsayım).
- **Bölme:** 1,77 ÷ 10 = 0,177 → tam gazın 0,177'si kadar ek güç lazım.
- **× 100:** kesri yüzdeye çevirmek (0,5 = %50 gibi) → 0,177 × 100 = **%17,7**
- Dikkat: "tam gaz ivmesi × 100" değil; **(1,77 ÷ 10) × 100**.

3 Kontrol: dengeleniyor mu?

- Pedal = %25 (düz yolda hızı tutmak için) + %17,7 (yokuş) = **%42,7**
- İtme: 10 × 42,7/100 = **+4,27**
- Hava/sürtünme kaybı: 0,025 × 100 = **−2,50**
- Yokuş: **−1,77**
- Toplam: 4,27 − 2,50 − 1,77 = **0** → hız değişmez ✓

Pedal	%100	%10	%1	%17,7
Her saniye hızlanma	10 km/h	1 km/h	0,1 km/h	1,77 km/h ✓

Kısaca: yokuş her saniye 1,77 km/h **çalıyor**; her %1 pedal 0,1 km/h **geri veriyor** → 1,77 ÷ 0,1 = **%17,7**.

Yokuşta döngü döngü: PID ve PID + FF

İleri besleme PID toplamına **dördüncü terim** olarak eklenir: **U = P + I + D + FF**

FF hataya bakmaz; bilinen bozucudan (yokuş) hesaplanır. Anti-windup kontrolünde de toplama FF dahil edilir.

A) Sadece PID (feedback) — yokuşu bilmiyor

	Döngü 1	Döngü 2
Sensör hızı	100	99,12
Hata e	0	0,88
P = 2 × e	0	1,77
D = 0,2 × Δe / 0,5	0	0,35
I = 0,1 × birikim	25	25,04
FF	—	—
U → pedal	%25	%27,2
0,5 s sonra hız	99,12 ↓	98,35 ↓

PID yokuşu ancak yavaşladıktan sonra fark ediyor; hep bir adım geride. Hız ~94 km/h'ye kadar düşüyor.

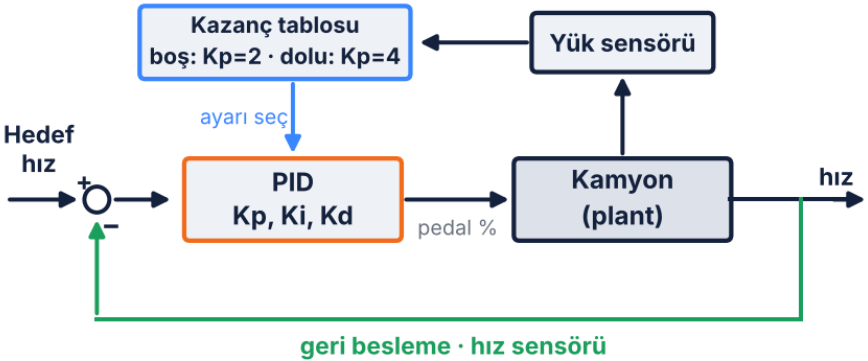
B) PID + ileri besleme (feedforward) — yokuş biliniyor

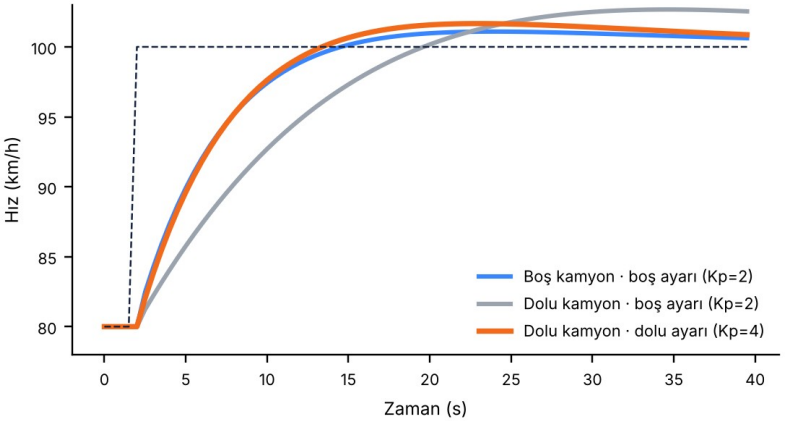
	Döngü 1	Döngü 2
Sensör hızı	100	100
Hata e	0	0
P = 2 × e	0	0
D = 0,2 × Δe / 0,5	0	0
I = 0,1 × birikim	25	25
FF = (1,77 ÷ 10) × 100	**17,7**	**17,7**
U → pedal	%42,7	%42,7
0,5 s sonra hız	100 ✓	100 ✓

Hata hiç oluşmuyor; PID'in işi kalmıyor. Yokuş tam bilinmezse kalan küçük farkı PID düzeltir.

Feedback: "Yavaşladım mı? O zaman gaza basayım." Feedforward: "Yokuş geliyor, şimdiden %17,7 fazla basayım."

Kazanç programlama örneği: boş ve dolu kamyon





- **Dolu kamyon 2 kat ağır** → aynı pedalla yarı ivme. Hedef 80 → 100 km/h.
- **Dolu kamyon + boş ayarı (gri):** hedefe ~16 s'de yetişir ve ~102,7'ye kadar aşar. Ayar ağır kamyona göre zayıf kalıyor.
- **Yük sensörü "dolu" der → tablo Kp, Ki, Kd'yi 2 katına çıkarır (turuncu):** ~10 s'de yetişir, boş kamyonla (mavi) aynı davranır.

Roketimizde kazanç programlama: Faz-1 (0–75 s, kalın hava, max-Q) ve Faz-2 (75–150 s, ince hava) için ayrı ayar. Seçimi saat yapıyor (t < 75 s mi?).

Kazanç programlama nasıl çalışır?

Yeni bir hesap yok, **bir seçim** var: PID formülü aynı kalır, sadece **Kp, Ki, Kd** duruma göre bir tablodan seçilir.

Programlama değişkeni

- (scheduling variable) = **ayarı seçtiren şey.** "Şu an hangi durumdayım?" sorusunun cevabı.
- **Ek sensör her zaman şart değil:** zaten bildiğimiz bir değer (zaman, hız) de olabilir.

Sistem	Programlama değişkeni	Ek sensör?
Kamyon	Yük (boş / dolu)	Yük sensörü
Uçak	İrtifa, hız	Var olan ölçümler
Bizim roket	**Saat: t < 75 s mi?**	**Yok, zamana bakıyoruz**

Kural: değişken → tablo → Kp, Ki, Kd. Örn. roket: t < 75 s ise G1, değilse G2.

Döngüde: normal PID'in başına tek adım eklenir

0. Duruma bak, ayarı seç ← kazanç programlama

1. Sensör ölçer → 2. Hata → 3. P, D, aday I → kontrol → U → pedal (aynen eskisi gibi)

Dolu kamyon, döngü 1	Boş ayarı (yanlış)	Dolu ayarı (doğru)
0. Seçilen ayar	Kp=2, Ki=0,1	Kp=4, Ki=0,2
Hata	20	20
P = Kp × e	**2 × 20 = 40**	**4 × 20 = 80**
U → pedal	%61	%100
0,5 s sonra hız	81,0 km/h	82,0 km/h

Aynı hataya doğru ayar **2 kat sert** tepki veriyor; ağır kamyon hedefe ~10 s'de (yanlış ayarla ~16 s) ulaşıyor.

Roketimizde: programlama değişkeni = zaman. Kp, Ki, Kd = G1 if t < 75 else G2. 75. saniyede sadece kazançlar değişir, birikim (I) aynen devam eder.

Özet: PID, ileri besleme ve kazanç programlama

PID'in yapısı hiç değişmiyor. İleri besleme U'ya **terim ekliyor**, kazanç programlama **Kp, Ki, Kd'yi seçiyor**.

Yapı	Ne değişiyor?	Denklem	Günlük örnek
Sadece PID (geri besleme)	Hiçbir şey	U = Kp·e + Ki·Σ + Kd·Δe/Δt	Cruise control (düz yol)
+ İleri besleme	U'ya yeni bir terim eklenir	U = P + I + D + FF	Yokuşu önceden görüp %17,7 ek pedal
+ Kazanç programlama	Formül aynı, Kp, Ki, Kd duruma göre seçilir	U = Kp(durum)·e + Ki(durum)·Σ + Kd(durum)·Δe/Δt	Boş / dolu kamyon

Roketimizde üçü birlikte:

$$U = Kp(t)\cdot e + Ki(t)\cdot \Sigma + Kd(t)\cdot de/dt + FF$$

Kp(t), Ki(t), Kd(t): 75. saniyeden önce G1, sonra G2 (GA bu 6 sayıyı arıyor) · **FF:** hedef ivme + yerçekimi + hava direnci (fizikten hesaplanıyor, ayarı yok)

Kısaca: PID hatayı düzeltir · İleri besleme bilineni baştan verir · Kazanç programlama duruma uygun ayarı seçer.

Tam döngü sırası (PID + FF + kazanç programlama)

Adım	Ne yapılıyor?	Formül
0	Ayarı seç (kazanç programlama) — YENİ	t < 75 s → G1, değilse G2
1	Sensör ölçer	ölçülen hız
2	Hata	e = hedef − ölçülen
3	P · Oransal	Kp · e
4	D · Türevsel	Kd · (e − e_önceki) / Δt
5	FF · İleri besleme — YENİ	bilinen bozucu (yokuş, yerçekimi)
6	Aday I · İntegral	Ki · (eski birikim + e · Δt)
7	Kontrol (anti-windup)	P + aday I + D + FF sınır içinde mi?
8	U toplam	U = P + I + D + FF
9	Sınırla	pedal %0–100 · roket gazı %40–100
10	Uygula Δt, sonra 1. adıma dön	araç 0,5 s · roket 0,1 s

Ne değişti?

- **Adım 0 (yeni):** döngünün en başında ayar seçilir.
- **Adım 5 (yeni):** FF hesaplanır.
- **Adım 7 ve 8:** toplama FF de girer.
- Diğer adımlar **aynen eskisi gibi**.

G1 / G2 ne zaman?

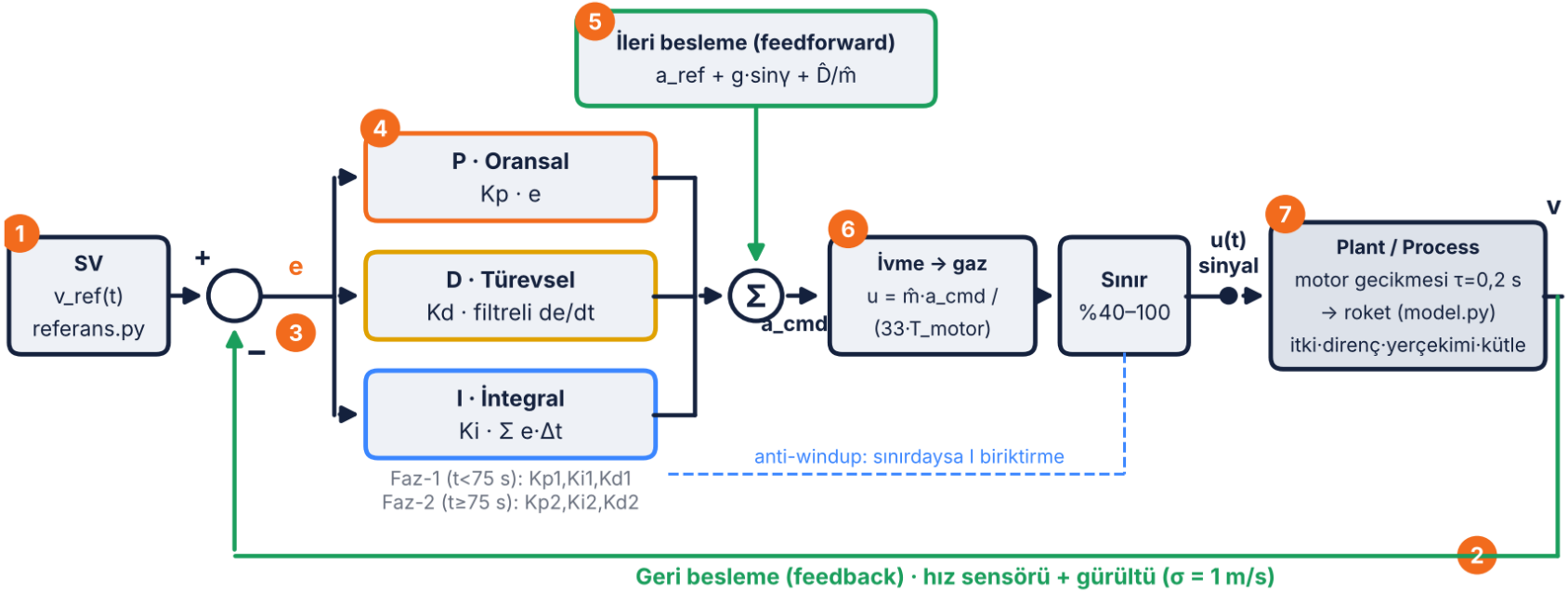
G1 · 0–75 s	G2 · 75–150 s

Sadece **iki** faz var. Adım 0 her döngüde (her 0,1 s, 1500 kez) kontrol edilir ama sonuç **bir kez**, 75. saniyede değişir.

Daha çok faz da yapılabilirdi; her faz 3 sayı daha demek.

Sıra: ayarı seç → ölç → hata → P → D → FF → aday I → kontrol → U → sınırla → uygula. Döngü her Δt'de tekrar.

Roketin kontrolcüsü nasıl hesaplanıyor?



Cruise control'den farkları: ① hedef sabit değil, veriden gelen bir eğri · ⑤ ileri besleme var · ⑥ çıkan ivme gaz koluna çevriliyor · Faz-1/Faz-2 için ayrı kazançlar.

Optimizasyonun (GA) aradığı 6 sayı: Faz-1 [Kp1, Ki1, Kd1] ve Faz-2 [Kp2, Ki2, Kd2]. Döngü her 0,1 s'de bir, 1500 kez çalışıyor.

PID yazılımda nasıl kuruldu? #src/model.py

Döngü kodu (sadeleştirilmiş) — numaralar diyagramla aynı

```
# her 0,1 s (k = 0 … 1499):
vref = VREF[ucus, k]                # 1 hedef hız
v_olc = v + gurultu[k]              # 2 sensör
e = vref - v_olc                    # 3 hata
Kp,Ki,Kd = G1 if t < 75 else G2     # 4 faz seçimi
I += e*DT  (sınırda değilse)        # 4 integral
d_f += ((e-e_prev)/DT - d_f)*0.17   # 4 türev (filtreli)
ff = aref + G0*sg + D_hat/m_hat     # 5 ileri besleme
a_cmd = ff + Kp*e + Ki*I + Kd*d_f   # Σ
u = m_hat*a_cmd / (33*T_motor)      # 6 ivme → gaz
u = clip(u, 0.40, 1.0)              # 6 sınır
T += (u*33*T_motor - T)*(DT/TAU)    # 7 motor gecikmesi
v += ((T - D)/m - G0*sg) * DT       # 7 roket hızlanır
```

1. **Hedef**
   referans.py'nin eğrisinden o saniyenin hedef hızı
2. **Ölç**
   Gerçek hız + 1 m/s sensör gürültüsü
3. **Hata**
   Hedef − ölçülen; + ise geride
4. **P, I, D**
   Faza göre kazanç seç; I biriktir, D'yi filtrele
5. **İleri besleme**
   Gereken ivmeyi baştan ver
6. **Gaza çevir**
   İvme → gaz kolu, %40–100 sınırı
7. **Roket**
   Motor gecikmesi + fizik → yeni hız

Excel'deki cruise control ile aynı sıra: ölç → hata → P, D, aday I → kontrol → toplam → sınırla → uygula. Fark: hedef eğri, ileri besleme ve fizik modeli.

Zor koşullar ekledik (senaryolar)

Her PID (şoför) **9 senaryoda** deneniyor: 3 yol (IFT-6, 7, 8) × her yolda 3 sürüş. Her sürüşte koşullar rastgele değişiyor:

- Roketin ağırlığı ±%3
- Hava direnci ±%30
- Motor gücü ±%3
- Bazı senaryolarda **bir motor arızalanıyor** (33 → 32)
- Hız sensörü **gürültülü** ölçüyor (±1 m/s)

Amaç: Sadece tek bir uçuşta değil, **beklenmedik koşullarda da** iyi çalışan şoförü bulmak.

1) Senaryoları üret (model.py)

```
def senaryolar(K=9, seed):
  ucus = [0,1,2, 0,1,2, 0,1,2]  # IFT-6,7,8
  fm   = uniform(0.97, 1.03)     # ağırlık ±%3
  fcd = uniform(0.70, 1.30)      # direnç ±%30
  ft   = uniform(0.97, 1.03)     # motor ±%3
  ariza = random() < 1/3         # arıza var mı?
  t_ariza = uniform(20, 140)     # arıza anı (s)
```

2) Simülasyonda kullan (simule)

```
m0  = M0 * fm                 # 5000 t ± %3
cdA = CD * AREA * fcd         # direnç katsayısı
n_aktif = 33 - (t >= t_ariza)  # 33 → 32
T_cmd = u * n_aktif * T_motor * ft
v_olc = v + normal(0, 1.0)    # sensör gürültüsü
```

seed sabit → her şoför aynı 9 sürüşte yarışıyor (adil karşılaştırma). Son testte daha önce hiç görülmemiş 200 yeni senaryo kullanıldı.

Koşullar: ne, gerçekte var mı, neden?

Koşul	Basitçe ne demek?	Gerçekte var mı?	Yazılımda ne yaptık?	Neden?
Yol	Takip edilecek hedef hız eğrisi	✓ Gerçek uçuş verisi	IFT-6, 7, 8 sırayla dağıtıldı	Tek uçuşa ezber olmasın
Ağırlık	Roket kalkışta kaç ton (~5000 t)	✓ Var	±%3 ağır / hafif	Yakıt dolumu farklı olabilir
Hava direnci	Havanın roketi ne kadar frenlediği	✓ Fizik kuvveti, her uçuşta var	Miktarı ±%30 değiştirildi	Direnç katsayısı (Cd) tahmin
Motor gücü	33 motorun ne kadar ittiği	✓ Var	±%3 güçlü / zayıf	Motorlar birebir aynı değil
Motor arızası	33 motordan biri durur	× Kullandığımız veride yok	Tamamen biz ekledik (~1/3 testte)	Beklenmedik duruma dayanıklılık
Sensör hatası	Hız ölçümü küçük hata yapar	Gerçek sensörlerde var	Biz ekledik (±1 m/s, her testte)	Ölçüm kusursuz değil

Direnç, ağırlık ve motor gücü zaten var; “ya tahminimiz yanlışsa?” diye değerlerini oynattık. Arıza ve sensör hatası PID’i zorlamak için bizim eklediğimiz durumlar.

9 test: her birinde ne oluyor?

Test	Yol	Ağırlık	Direnç	Motor gücü	Arıza	Bu testte ne oluyor? (basitçe)
1	IFT-6	%2,6 ağır	%12 az	%2,9 güçlü	yok	Ağır ama güçlü; normale yakın, kolay uçuş
2	IFT-7	%1,9 ağır	%5 az	%1,1 güçlü	126,7. s	Sona doğru bir motor duruyor; PID açığı kapatmalı
3	IFT-8	%3 hafif	%28 az	%0,9 güçlü	132,1. s	Hafif, az dirençli: fazla hızlanmaya yatkın; sonda motor kaybı
4	**IFT-6**	**%2,1 ağır**	**%22 az**	**%1,1 güçlü**	**62,9. s**	**En zor: motor erken, max-Q yakınında bozuluyor**
5	IFT-7	%2,8 hafif	%10 fazla	%0,7 zayıf	yok	Hafif ama zayıf motor ve fazla direnç; dengeli
6	IFT-8	%1,4 ağır	%9 fazla	%2,2 zayıf	yok	Geride kalmaya yatkın; PID gazı artırmalı
7	IFT-6	%1,9 hafif	%7 fazla	%1,3 güçlü	yok	Hafif ve güçlü; kolay uçuş
8	IFT-7	%2,2 ağır	%7 az	normal	yok	Normale çok yakın, “standart” uçuş
9	IFT-8	normal	%30 fazla	%1,1 zayıf	yok	Hava en çok frenliyor; max-Q’ya en çok yaklaşılan test

Kolay testler (1, 7, 8), geride bırakan (6, 9), fazla hızlandıran (3), motor arızalı (2, 3, 4). Bulunan PID ayarı hepsinde iyi → güvenilir.

Seed nedir? Test kâğıdının numarası

Basitçe

- **seed = 0** demek: “9 testi bir kere belirle, **hep aynısını** kullan.”
- **Sınav benzetmesi:** öğrenciler = PID ayarları, soru kâğıdı = 9 test, seed = kitapçık numarası. Herkese aynı kâğıt → adil karşılaştırma.
- seed değişirse 9 testteki durumlar değişir; **yol sırası** (IFT-6, 7, 8…) değişmez.

Nerede tanımlı?

```
# deneyler.py, KayitliAmac
super().__init__(C.N_SCEN, seed=0)
# model.py, AmacFonksiyonu
def __init__(self, K=9, seed=0):
    self.sc = senaryolar(K, seed)
# → np.random.default_rng(0)
```

seed = 0 ile seed = 1 karşılaştırması

	seed = 0 (eğitim)	seed = 1
Test 1 ağırlık	%102,6	%97,2
Test 1 direnç	%88	%82
Arızalı testler	2, 3, 4	3, 5
Bulunan ayarın J puanı	**0,6592**	**0,6574**

Neredeyse aynı puan → ayar seed = 0’ın sorularını **ezberlememiş**.

Kodda başka seed’ler (karıştırma)

Seed	Ne için?
0	Eğitimdeki 9 test kâğıdı
0 + 999	Sensör hatası (her seferinde aynı)
12345	Son test: 200 yeni senaryo
s = 1…10	Algoritmanın 10 koşusu (test kâğıdı değil)

Seed hangi değer olursa olsun, durumlar hep aynı sınırlarda kalır: ağırlık ±%3, direnç ±%30, motor ±%3, arıza ~1/3 testte.

Puanlama sistemi (J) kurduk

Her sürüşte gözlemci deftere not alıyor. Notlar tek bir **J puanına** dönüşüyor (**küçük = iyi**):

Terim	Sorduğu soru
IAE / 300	Hedef hızdan ne kadar saptın?
(Δv_son / 10)²	Son saniyede hızın doğru mu?
20 · max-Q aşımı	Max-Q sınırını aştın mı? Aştıysan büyük ceza.
0,5 · yakıt / 2700 t	Ne kadar yakıt harcadın?
0,003 · TV	Gaz kolunu ne kadar oynattın?

9 sürüşün ortalaması, şoförün notu oluyor.

Defter ve puan (model.py)

```
# uçuş sırasında, her 0,1 s:
iae += abs(v - vref) * DT   # sapma alanı
tv  += abs(u - u_prev)      # gaz oynaması
qmax = max(qmax, q)         # en yüksek basınç

# uçuş bitince: maliyet(r)
J = ( iae / 300
    + ((v_son - v_ref_son) / 10)**2
    + 20 * max(0, qmax/q_lim - 1)
    + 0.5 * yakit / 2700e3
    + 0.003 * tv )
return J.mean()             # 9 sürüş ortalaması
```

Bölenler (300, 10, 2700 t) terimleri aynı ölçeğe getiriyor. 20 çarpanı: max-Q aşımını en ağır cezaya çeviriyor (ceza yöntemi).

05

Yazılım: Python dosyaları #src

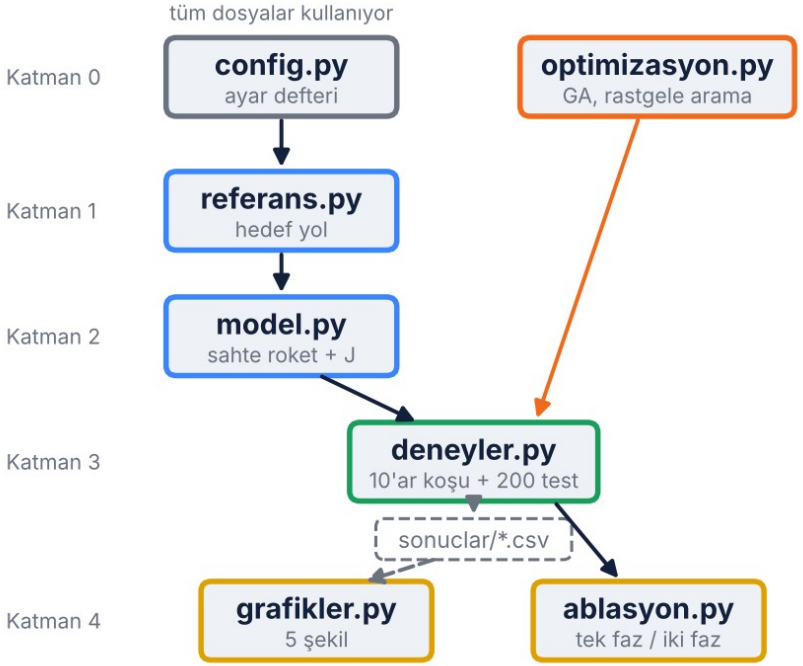
Hangi dosya ne yapıyor, birbirini nasıl kullanıyor: config, referans, model, optimizasyon, deneyler.

Python dosyaları: kim ne iş yapıyor?

Dosya	Benzetme	Ne işe yarıyor?	Import ettiği proje dosyaları
config.py	Ayar defteri	Bütün sabit sayılar: 5000 t, 33 motor, Isp, Δt = 0,1 s, 75 s, PID sınırları, 30 × 60 bütçe, 9 senaryo	— (hiçbiri)
referans.py	Yol haritası	CSV'yi oku, temizle, yumuşat; hedef hız, ivme, eğim, max-Q	config
model.py	Sahte roket + puan makinesi	Roketi 9 testte uçur, J puanını ver	config, referans
optimizasyon.py	Arama yöntemleri	GA ve rastgele arama; roketi bilmez	— (sadece numpy)
deneyler.py	Deney yöneticisi	Her algoritma 10 koşu, istatistik testi, 200 senaryoda test → sonuclar/*.csv	config, model, optimizasyon
grafikler.py	Ressam	CSV'lerden 5 şekil çizer (bildirideki Şekil 1–5)	config, model, referans, deneyler
ablasyon.py	Ek deney	Tek faz / iki faz karşılaştırması	config, model, optimizasyon, deneyler

Her dosya sadece kendinden alttakileri kullanıyor. Senin çalıştırdığın: önce deneyler.py, sonra grafikler.py.

Yol haritası: bağlantılar ve çalışma sırası



Adım 1: python deneyler.py

- config okunur; model import edilir → referans CSV'yi okur, 3 hedef yol hazır
- GA × 10, rastgele × 10 → puan makinesine soruyorlar
- İstatistik testi, en iyi ayar 200 yeni senaryoda test
- sonuclar/ klasörüne CSV ve JSON yazılır

Adım 2: python grafikler.py

- deneyler'den sadece 3 sabit alınır (optimizasyon tekrar çalışmaz)
- sonuclar/*.csv okunur → 5 şekil sonuclar/sekiller/ klasörüne

config her yerde: tek başına bir adım değil, hep açık bir ayar defteri.

optimizasyon roketi bilmiyor: model'i deneyler bağlıyor: O.ga(f, ...). Aynı GA İrem'in modelinde de çalışır.

Sıra: config → referans → model → (optimizasyon) → deneyler → grafikler. Önce deneyler çalışmazsa grafikler CSV bulamaz.

config.py: ayar defteri (1/2)

Hesap yapmaz; sadece sayıları ve dosya yollarını tanımlar. Bütün dosyalar buradan okur.

1) Dosya yolları: “dosyalar nerede?”

Satır	Basitçe
KOK	Proje klasörü (src’nin bir üstü)
_ADAYLAR	CSV’nin aranacağı 4 klasör
VERI_KLASORU	ift8_results.csv hangisindeyse onu seç
raise FileNotFoundError	Hiçbirinde yoksa açık hata mesajı
UCUSLAR	ift6, ift7, ift8
SONUC / SEKIL_KLASORU	sonuclar/ ve sonuclar/sekiller/

2) Fiziksel sabitler

Satır	Değer	Anlamı
G0	9,81 m/s^2	Yerçekimi
RHO0	1,225 kg/m^3	Yerdeki hava yoğunluğu
H_SCALE	8500 m	Havanın incelme ölçeği

3) Araç: roketin kendisi

Satır	Değer	Anlamı
M0	5000 t	Kalkış kütlesi (booster + ship)
DIAMETER / AREA	9 m / 63,6 m^2	Çap ve önden alan
CD	0,5	Direnç katsayısı (tahmin, ±%30)

4) Raptor motorları

Satır	Değer	Anlamı
N_ENG	33	Motor sayısı
MDOT_ENG	705 kg/s	Motor başına yakıt
ISP_SL / ISP_VAC	327 / 350 s	Yerde / uzayda verim
THROTTLE_MIN	0,40	Gaz en az %40
TAU_ENGINE	0,20 s	Motor gecikmesi

705 × 327 × 9,81 = 2,26 MN → bir motorun itkisi bu üç sayıdan çıkıyor. PyCharm’daki “CSV bulunamadı” düzeltmesi 1. bölümde.

config.py: ayar defteri (2/2)

5) Uçuş fazları

Satır	Değer	Anlamı
T_END	150 s	İncelenen süre (ayrılmadan önce)
T_SWITCH	75 s	Faz-1 → Faz-2 geçişi

6) Simülasyon

Satır	Değer	Anlamı
DT	0,1 s	Döngü süresi (Δt) → 1500 adım
SENSOR_NOISE	1,0 m/s	Hız sensörü hatası
Q_MARJ	1,05	Max-Q sınırı = 1,05 × gerçek (≈ 23 kPa)
P_MOTOR_ARIZA	1/3	Testlerin üçte birinde arıza

Kullanım ipucu: bir şeyi denemek için sadece buradaki sayıyı değiştir.
Örn. T_SWITCH = 90.0 yaz → python deneyler.py. Diğer dosyalara dokunma.

7) PID arama sınırları

Satır	Değer	Anlamı
LOWER	[0, 0, 0, 0, 0, 0]	En küçük değerler
UPPER	[2, 0,5, 1, 2, 0,5, 1]	Kp ≤ 2, Ki ≤ 0,5, Kd ≤ 1
PARAM_ADLARI	Kp1 … Kd2	Tablolardaki sütun adları

GA 6 sayıyı sadece bu kutunun içinde arıyor.

8) Optimizasyon bütçesi

Satır	Değer	Anlamı
POP × ITER	30 × 60	1800 deneme / koşu
N_RUNS	10	Her algoritma 10 kez
N_SCEN	9	Her ayar 9 testte

Bütçe GA ve rastgele arama için **aynı** → adil karşılaştırma.

Özet: dosyalar nerede · doğa sabitleri · roket · motorlar · zaman · simülasyon · PID nerede aranacak · ne kadar aranacak.

model.py kod rehberi: dosyanın yapısı

Parça	Benzetme	Görevi
Baştaki tablolar (VREF…)	Kopya kâğıdı	Hedef yol her 0,1 s için hazır
senaryolar()	Test listesi	9 zor sürüşün koşulları
_isp()	Motor verimi	Yükseldikçe motor güçlenir
simule()	Yol + şoför + gözlemci	150 s uçuş, defter tutma
maliyet()	Karne	Defter → tek puan J
AmacFonksiyonu	Puan makinesi	6 sayı gir → J çıksın
if __name__ == "__main__"	Deneme	4 örnek ayarla kontrol

Baştaki tablolar

```
N_ADIM = int(round(150/0.1)) + 1
_tg = np.arange(N_ADIM) * 0.1
VREF = [R.v_ref(_tg) for R in REFS]
AREF = [R.a_ref(_tg) for R in REFS]
SG   = [R.sin_gamma(_tg) for R in REFS]
QMAX_REF = [R.q_max for R in REFS]
```

Satır	Ne yapıyor?	Sonuç
N_ADIM, _tg	Kaç an var, saatleri ne? (_tg = time grid, bir değişken)	1501 an: 0; 0,1; … 150 (çitteki direk gibi +1)
VREF, AREF, SG	Her uçuşa her an için: hedef hız, hedef ivme, eğim	3 × 1501 tablo; QMAX_REF = 3 sayı (21,8 · 22,3 · 21,8 kPa)

Tablolar bir kez hesaplanıyor; simülasyon milyonlarca kez sadece tablodan okuyor → çok hızlı.

senaryolar(K, seed): test listesini hazırlar

Kod satırı	Basitçe	Sonuç (K = 9, seed = 0)
def senaryolar(K, seed)	K = kaç test (soru sayısı), seed = hangi liste (kitapçık no)	Eğitim (9, 0) · son test (200, 12345)
r = np.random.default_rng(seed)	Sayı makinesi kur, adı r; seed sabit → hep aynı sayılar	—
ariza = r.random(K) < 1/3	0–1 arası 9 sayı; 0,33'ten küçükse arıza var	0,64 · 0,27 · 0,04 · 0,02 · … → Test 2, 3, 4
ucus = np.arange(K) % 3	0…8 sıra no, 3'e bölümden kalan → yolları sırayla dağıt	IFT-6, 7, 8, 6, 7, 8, 6, 7, 8
fm = r.uniform(0.97, 1.03, K)	Ağırlık çarpanı ±%3 (factor mass)	1,026 → 5130 t
fcd = r.uniform(0.70, 1.30, K)	Hava direnci çarpanı ±%30 (factor Cd)	0,88 → %12 az
ft = r.uniform(0.97, 1.03, K)	Motor gücü çarpanı ±%3 (factor thrust)	1,029 → %2,9 güçlü
t_ariza = np.where(ariza, 20–140, inf)	Arıza varsa saniyesi, yoksa "hiçbir zaman" (∞)	yok · 126,7 · 132,1 · 62,9 · yok …
return dict(…)	Etiketli paket (çekmeceler) olarak teslim et	simule() içinde sc["fm"] gibi açılıyor

Önemli: Bu fonksiyon roketi uçurmuyor, sadece "hangi koşullarda uçacak?" listesini dolduruyor. Yol sırası seed'e bağlı değil; seed değişirse sadece ağırlık, direnç, motor ve arıza değerleri değişir.

Sınav benzetmesi: K = soru sayısı, seed = kitapçık numarası. Herkese (her PID ayarına) aynı kâğıt → adil karşılaştırma.

_isp() ve simule(): motor verimi ve uçuş

_isp(h): roket yükseldikçe motor daha verimli

```
return 350 - (350 - 327) * exp(-h / 8500)
```

Yükseklik	Isp	Motor başına itki
0 km (yer)	327 s	2,26 MN
8,5 km	341 s	≈ 2,36 MN
55 km (150. s)	≈ 350 s	2,42 MN (~%7 fazla)

- **Isp:** “litre başına km” gibi motor verimi. Yerde hava egzozu geri itiyor (327), uzayda itmiyor (350).
- **Kullanıldığı 3 yer:** ① motor itkisi ② ivme → gaz kolu ③ yanan yakıt.
- Olmasaydı sahte roket yüksekte gerçekte olduğundan yavaş hızlanırdı.

simule(): 150 s uçuş, 3 bölüm

Bölüm	Ne yapıyor?
1. Hazırlık	Ayarları G1/G2’ye ayır; test koşullarını uygula (m0 = 5000 t × fm …); roketi, PID hafızasını ve defteri sıfırla; sensör hatalarını üret
2. Döngü (1500 kez, her 0,1 s)	Ayar seç (t < 75?) → hedefi oku → hava, Isp, itki → ölç, hata → D (filtreli) → ileri besleme + PID → gaz kolu (%40–100) → anti-windup → arıza (33→32) → motor gecikmesi → Newton → deftere yaz → yeni hız, yükseklik, kütle
3. Sonuç	Defteri teslim et: iae, tv, qmax, amax, yakıt, son hız, sınır

Önemli incelik: aynı anda “ayar sayısı × test sayısı” kadar roket uçuyor (örn. 30 × 9 = 270); her değişken bir matris.

maliyet() ve AmacFonksiyonu: karne ve puan makinesi

maliyet(): defter → tek puan J (küçük = iyi) · Test 1, en iyi ayar

Terim	Defter	Hesap	Puan
iae / 300	30,7	30,7 ÷ 300	0,102
(Δv_son / 10)²	−0,01 m/s	(−0,01 ÷ 10)²	0,000
20 × max-Q aşımı	21,8 / 22,9 kPa	aşım yok	0
0,5 × yakıt / 2700 t	2816 t	0,5 × 2816 ÷ 2700	0,522
0,003 × tv	15,7	0,003 × 15,7	0,047
Toplam			**J = 0,671**

- 9 testin ortalaması → **J = 0,659** (en kötü: Test 4, 0,711).
- **Dürüst not:** puanın ~0,50'si yakıt; ama yakıt ayardan pek etkilenmiyor. Farkı asıl **sapma (iae)** ve **gaz oynaması (tv)** belirliyor.

AmacFonksiyonu = puan makinesi

6 sayı gir → J çıksın.

Örn: [0,23 · 0,08 · 0 · 0,15 · 0,08 · 0] → **0,659**

- **Kurulurken (bir kez):** 9 testlik listeyi hazırla, sayacı (n_eval) 0'la.
- **Her soruda:** sayacı artır → roketi 9 testte uçur (simule) → puanla (maliyet) → J'yi ver.
- **Neden?** GA roketi bilmiyor; sadece bu makineye ayar verip puan alıyor. Sayaç 1800 denemeyi geçmemeyi kontrol ediyor.

Bölenler (300, 10, 2700 t) terimleri aynı ölçeğe getirir; 20 çarpanı max-Q aşımını en ağır cezaya çevirir.

Deneme bölümü: çıktı nasıl okunur?

python model.py → 4 elle seçilmiş ayarla puan makinesinin mantıklı sonuç verip vermediğini ve hızını kontrol eder (optimizasyonda kullanılmaz).

Çıktı satırı	1. Yumuşak (0,5 · 0,05 · 0,05)	2. Orta (1 · 0,2 · 0,2)	3. PID kapalı (0 · 0 · 0)	4. Çok sert (4 · 1,5 · 1,5)
J (puan)	0,75 ✓	0,94	97,9 ×	1,79
iae (sapma, m)	35	40	4956	95
qmax / q_lim (Pa)	21.936 / 23.064	21.987 / 23.064	22.100 / 23.064	22.125 / 23.064
tv (gaz oynaması)	46	102	0,6	325
amax (en yüksek ivme, g)	2,1	2,2	2,1	2,8

v_son – ref tablosu (4 × 9)

- Satır = ayar, sütun = test; 150. saniyedeki hız hatası (m/s).
- Ayar 1, 2, 4: hata ±1 m/s içinde ✓
- Ayar 3 (PID kapalı): −105 ile +134 m/s × (düzeltme yok)

Okuma kuralı

- Her satırdaki 4 sayı = 4 ayar, aynı sırayla.
- PID kapalıda max-Q **aşılmıyor**; 97,9 puanın sebebi büyük son hız hatası ve sapma.
- “30 aday: 0,13 s” → bir nesil 0,13 saniyede puanlanıyor.

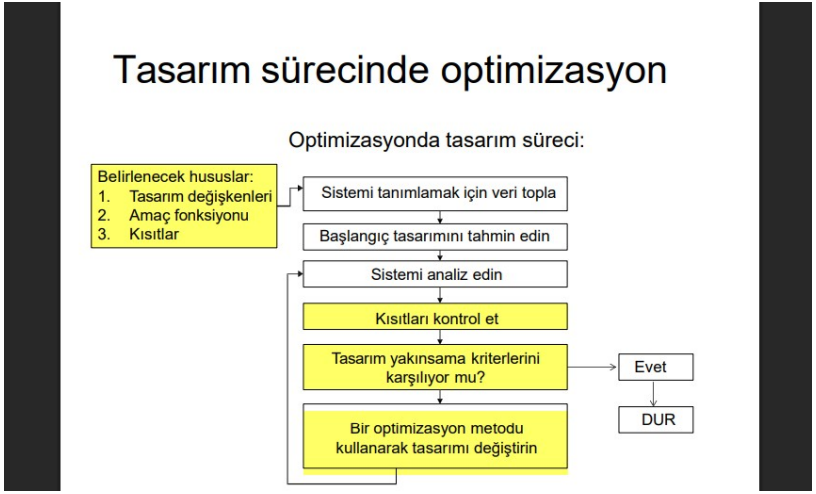
Sonuç: makine mantıklı puan veriyor (PID’siz çok kötü, sert kötü, yumuşak iyi); GA bundan da iyisini (0,659) buluyor.

06

Optimizasyon: Genetik Algoritma

#src/optimizasyon.py · Ders şemasından koda: set, turnuva, çaprazlama, mutasyon, nesil ve koşu.

optimizasyon.py: ders şemasıyla eşleşme



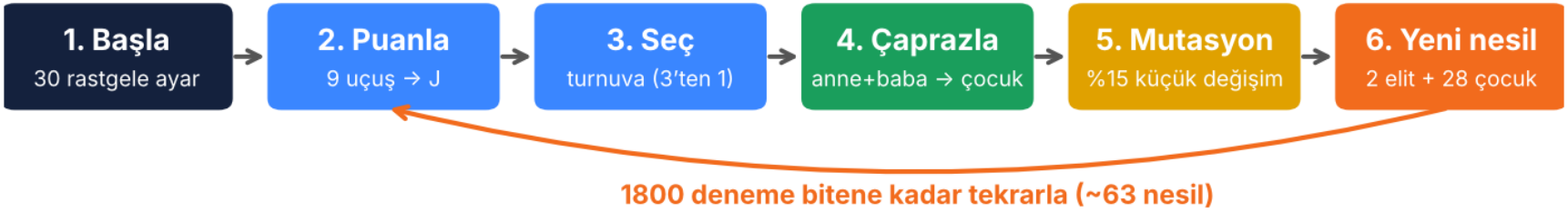
Ders notu: "Tasarım sürecinde optimizasyon"

Şemadaki kutu	Projede	Dosya
Tasarım değişkenleri	6 kazanç: Kp1 Ki1 Kd1 Kp2 Ki2 Kd2	config.py
Amaç fonksiyonu	J = sapma + son hız + yakıt + gaz oynaması + ceza	model.py
Kısıtlar	Max-Q ≤ 1,05 × referans; kazanç sınırları	config / model
Veri topla	Kaggle IFT-6/7/8 → referans hız	referans.py
Başlangıç tasarımı	30 rastgele ayar	optimizasyon.py
Sistemi analiz et	Her ayarla 9 sahte uçuş	model.py
Kısıtları kontrol et	Max-Q aşılırsa J'ye büyük ceza	model.py
Yakınsama? → DUR	1800 deneme bitti mi?	optimizasyon.py
Tasarımı değiştir	Genetik Algoritma (GA)	optimizasyon.py

3 fark: kısıt ayrı kutu değil, J'ye ceza · durma kuralı sabit bütçe (adil yarış) · başlangıç tek tahmin değil, 30 ayarlık grup.

Genetik Algoritma (GA): en iyi kek tarifini bulmak

Benzetme: 30 kek tarifin var, her tarif 6 sayı (Kp1 … Kd2). Her keki pişirip puanlıyorsun (J, **küçük = iyi**). İyi tarifleri karıştırıp yeni tarifler deniyorsun, en iyileri hep saklıyorsun.



Benzetmede	Projede
Tarif (6 sayı)	Bir PID ayarı: Kp1, Ki1, Kd1, Kp2, Ki2, Kd2
Keki pişirip tatmak	9 sahte uçuş yapıp J hesaplamak (model.py)
30 tarif · her turda 2 şampiyon + 28 yeni	POP = 30 · elit = 2 · 28 çocuk
Birkaç tur	~63 nesil = 1800 deneme (≈ 9 saniye)

Tek cümlede GA: iyileri seç → karıştır → biraz değiştir → en iyileri sakla → tekrarla.

Bir setin sınavı: 9 uçuş → 1 J

#	Yol	O uçuştaki durum	Puan
1	IFT-6	normal	0,671
2	IFT-7	126,7. s motor arızası	0,680
3	IFT-8	132,1. s motor arızası	0,649
4	IFT-6	62,9. s motor arızası	0,711
5	IFT-7	%10 fazla hava direnci	0,637
6	IFT-8	%2 zayıf motor	0,648
7	IFT-6	%2 hafif roket	0,647
8	IFT-7	%2 ağır roket	0,660
9	IFT-8	%30 fazla hava direnci	0,629
		Ortalama = setin J'si	**0,659**

Tek uçuşun puanı = 5 maddelik ceza toplamı

Madde	Neyi ölçer?	1. uçuş
Sapma (IAE) / 300	Yol boyunca hız hatası	0,102
(Son hız hatası/10)²	150. s'de doğru hızda mı?	≈ 0
Max-Q cezası	Basınç sınırı aşıldı mı?	0
0,5 · yakıt / 2700 t	Ne kadar yakıt yandı?	0,522
0,003 · gaz oynaması	Gaz kolu ne kadar oynadı?	0,047
Toplam		**0,671**

- J sadece hataların ortalaması **değil**: hız takibi + güvenlik + yakıt + motor yorgunluğu.
- Yakıt en büyük pay ama ayardan ayara neredeyse değişmez; setleri ayıran **sapma** ve **gaz oynaması**.

Benzetme: 9 sınavın ortalaması = karne notu. Her setin tek bir J'si var; küçük J = iyi set.

Bütçe: 1800 deneme hakkı

1 deneme = bir seti 9 uçuşa sokup J'sini hesaplamak. **Bütçe** = pop × iters = 30 × 60 = **1800 deneme** = 16.200 sahte uçuş ≈ 9 saniye.

Ne zaman?	Harcanan	Toplam
Başlangıç: 30 rastgele set	30	30
1. nesil: 28 çocuk	28	58
2. nesil: 28 çocuk	28	86
…	…	…
63. nesil	28	1794
Son nesil: kalan hak	**6**	**1800 → DUR**

Neden 30 değil 28?

- 2 şampiyon tekrar uçurulmaz, eski J'leri kullanılır → her nesil 28 deneme yer.
- Bu yüzden 60 değil ~64 nesil çıkar.

Neden bütçe var?

- **Adil yarış:** GA ve Rastgele Arama'ya aynı hak.
- Ders şemasındaki "Yakınsama? → DUR" kutusu: while n_ev < butce

Bütçe bitince durulur; o ana kadarki en küçük J'li set = sonuç (6 kazanç).

GA örnekle: tek düğme (Kp), 4 tarif

Tarif	Ayar (Kp)	Puan (J)
A	0,2	0,80
B	**0,9**	**0,68 ← en iyi**
C	1,4	0,72 ← ikinci
D	1,8	0,90 ← en kötü

- **Dikkat:** 0,9 bir puan değil, **ayarın kendisi** (düğmenin konumu). Puan J sütununda.
- **Adım 1:** Rastgele 4 Kp seç, her biriyle uçur, puanla.
- **Adım 2 · Seçim (turnuva):** torbadan rastgele 3 tarif çek (ör. A, B, D) → en iyisi **B** kazanır = anne. Bir daha çek (B, C, D) → **C** = baba. İyiler sık seçilir, zayıflara da küçük şans kalır.
- **Adım 3 · Çaprazlama:** anne 0,9 ile baba 1,4 karışır → çocuk **1,10**.
- **Adım 4 · Mutasyon:** küçük ihtimalle biraz değişir → 1,10 → **1,18**.
- **Adım 5 · Yeni nesil:** B ve C **olduğu gibi kalır** (elitizm: şampiyon kaybolmaz). A ve D atılır, yerlerine çocuklar gelir → 0,9 · 1,4 · 1,18 · 1,05.
- **Adım 6:** Tekrar puanla… Birkaç nesil sonra hepsi en iyi Kp'nin etrafında toplanır.

Projede aynısı: 1 yerine 6 sayı, 4 yerine 30 ayar, birkaç yerine ~63 nesil.

Turnuva: anne ve baba nasıl seçilir?

	Rastgele çekilen 3 set	J'leri (örnek)	Kazanan = J'si en küçük
1. turnuva	Set 1 · Set 3 · Set 5	0,84 · 1,95 · 0,72	Set 5 → anne
2. turnuva	Set 3 · Set 7 · Set 5	1,95 · 0,69 · 0,72	Set 7 → baba

Bilmen gerekenler

- Turnuvalar hep **eski 30 setten** yapılır; J'leri zaten belli, yollar tekrar uçulmaz.
- Aynı set iki turnuvaya da girebilir (Set 5 gibi).
- Anne ile baba aynı set çıkabilir → çocuk kopya olur, farkı sadece mutasyon yaratır.
- Yeni çocuklar turnuvaya katılmaz: henüz J'leri yok.

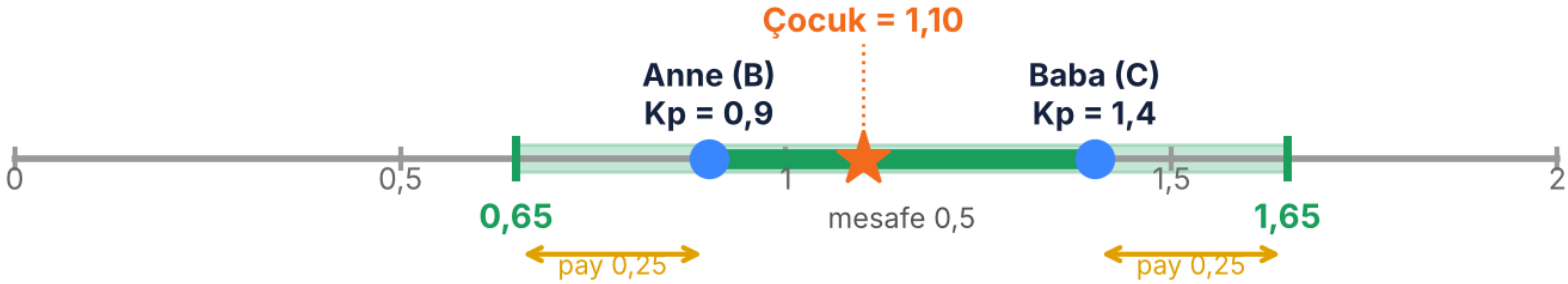
Neden turnuva?

- İyi setler **sık** kazanır → iyi özellikler çoğalır.
- Kötü setlerin de **küçük** şansı kalır → çeşitlilik korunur, GA hemen tek noktaya kilitlenmez.
- 1 çocuk = 2 turnuva · 1 nesil = 28 çocuk = 56 turnuva

Turnuvadan sonra: çaprazlama (anne + baba → çocuk).

Çaprazlama: anne + baba → çocuk

Benzetme: Çocuğun boyu genelde anne ile babanın arasında olur, bazen biraz kısa ya da uzun. Çocuğun Kp'si de ebeveynlerin Kp'leri **arasından (biraz da dışından)** seçilir.



Adım	Hesap
1. Mesafe	1,4 − 0,9 = 0,5
2. Pay (yarısı, α = 0,5)	0,5 × 0,5 = 0,25
3. Aralık	0,9 − 0,25 = 0,65 … 1,4 + 0,25 = 1,65
4. Rastgele seç	çocuk = 1,10

Sayı	Anne	Baba	Çocuk aralığı
Kp1	0,9	1,4	0,65 – 1,65
Ki1	0,10	0,30	0,00 – 0,40 (0'ın altı yok)
Kd1	0,20	0,20	0,20 (aynı → değişmez)
…	…	…	her sayı ayrı ayrı

Neden dışarı pay? En iyi Kp belki 0,7: ebeveynlerin dışında. Pay olmasa oraya hiç bakılmazdı.

Çaprazlama 6 sütunda: anne Set 3 × baba Set 8

10 çocuktan 9'unda karıştır, 1'inde çocuk annenin kopyası. Her sütun için aynı 5 adım:

1. Alt alta yaz	2. Fark = büyük − küçük	3. Pay = fark × 0,5	4. Aralık = küçük − pay … büyük + pay	5. Aralıktan rastgele seç

Sayı	Anne (Set 3)	Baba (Set 8)	Fark	Pay	Aralık	Çocuk
Kp1	0,90	1,40	0,50	0,25	0,65 – 1,65	1,10
Ki1	0,10	0,30	0,20	0,10	0,00 – 0,40	0,22
Kd1	0,20	0,20	0	0	0,20 (değişemez)	0,20
Kp2	0,60	0,40	0,20	0,10	0,30 – 0,70	0,52
Ki2	0,30	0,10	0,20	0,10	0,00 – 0,40	0,17
Kd2	0,05	0,25	0,20	0,10	−0,05 – 0,35	0,12

Kim anne kim baba fark etmez (küçük/büyük'e bakılır) · Ortalamayla da olur: ortalama ± fark → (0,90+1,40)/2 = 1,15 ± 0,50

Benzetme: anne 160 cm, baba 180 cm → çocuk büyük ihtimalle arada (170), bazen biraz kısa ya da uzun.

Mutasyon: çocuğun bir sayısını biraz değiştirmek

Benzetme: Çocuk ebeveynlere benzer ama bazen ailede kimsede olmayan bir özellikle doğar. Mutasyon, **kimsenin denemediği değerleri** de denetir.

	Kp1	Ki1	Kd1	Kp2	Ki2	Kd2
Çaprazlama sonrası	1,10	0,22	0,20	0,80	0,15	0,05
Mutasyon sonrası	**1,10**	**0,22**	**0,27 ↑**	**0,80**	**0,15**	**0,05**

Hangisi değişir? %15 ihtimal

Sayı	Bilgisayarın ürettiği (0–1)	< 0,15 ?
Kp1	0,62	Hayır → aynı
Kd1	0,08	Evet → değişir
Kp2	0,93	Hayır → aynı
…	…	…
Sonuç	6 × 0,15 ≈ her çocukta 1 sayı	%38'inde hiç yok

Ne kadar değişir? Aralığın ~%10'u

Düğme	Aralık	Tipik değişim
Kp	0 – 2	≈ 0,2
Ki	0 – 0,5	≈ 0,05
Kd	0 – 1	≈ 0,1

Değişim çoğu zaman küçük, nadiren büyük (çan eğrisi). 0'ın altına düşerse 0, üst sınırı aşarsa sınır yapılır.

Neden %15? Kural: ihtimal ≈ 1 ÷ sayı adedi = 1/6 ≈ %15 → her çocukta yaklaşık 1 sayı değişsin.

Önce doldur, sonra oynat: mutasyon + sınır kontrolü

	Kp1	Ki1	Kd1	Kp2	Ki2	Kd2
1. Çaprazlama: 6'sı da doldu	1,10	0,22	0,20	0,52	0,17	0,12
2. Mutasyon: bazıları biraz oynadı	**1,10**	**0,22**	**0,27 ↑**	**0,52**	**0,17**	**0,12**

Kaç sayı değişir? (her sayı ayrı ayrı %15)

Durum	Ne sıklıkla?
Hiçbiri değişmez	~%38
1 sayı değişir	~%40
2 sayı değişir	~%18
3 veya daha fazla	~%4

- Mutasyon boş yer doldurmaz: **dolu sayıya küçük ekleme/çıkarma** yapar (0,20 + 0,07).
- Ortalamada ~1 sayı değişir; her seferinde aynı değil.

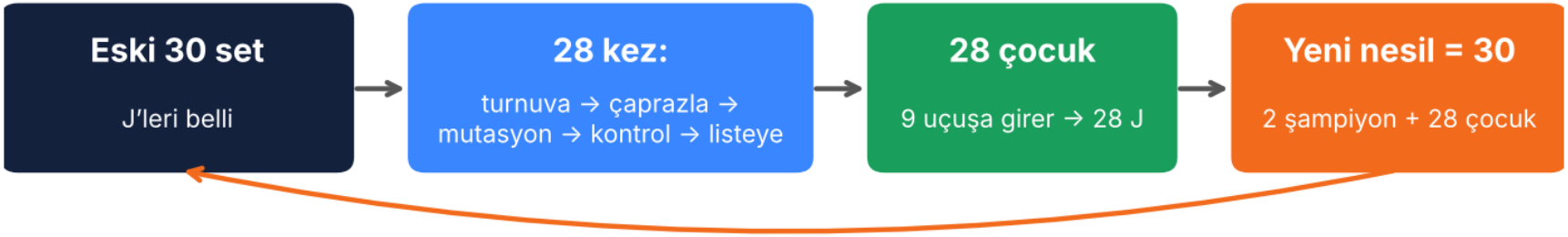
Sınır kontrolü: aralıkta kal

Durum	Örnek	Sonuç
Alttan taştı	Kd2 = −0,03	0
Üstten taştı	Kp1 = 2,15	2
Aralıkta	Ki1 = 0,22	dokunma
Sınırlar	Kp 0–2 · Ki 0–0,5 · Kd 0–1	

- Termostat gibi: 35 yazsan da 30'a çeker.
- Neden? Negatif kazanç PID'i ters çalıştırır → roket kontrolden çıkar. Kod: np.clip(c, lb, ub)

Bir çocuk = turnuva → çaprazlama (6'sı dolar) → mutasyon (%15 oynat) → sınır kontrolü → listeye.

Bir neslin hikâyesi: şampiyon ≠ turnuva kazananı



	Şampiyon (elit)	Turnuva kazananı (anne/baba)
Nasıl seçilir?	30 setin hepsine bak, J'si en küçük 2 set (ör. Set 2 ve Set 8)	Rastgele 3 set çek, içlerinden en iyisi
Kaç tane?	Her nesilde 2	Her çocuk için 2
Ne olur?	Olduğu gibi yeni nesle geçer, tekrar uçurulmaz	Çocuk yapmakta kullanılır, kendisi geçmez
Benzetme	Takımın en iyi 2 oyuncusu: kadroda kalır	Anne-baba: çocuk yetişince emekli olur

Çocuklar listede bekler, 28'i tamamlanınca birlikte 9 uçuşa girer. Bir set hem şampiyon hem ebeveyn olabilir.

ga() kodu adım adım (1/2): hazırlık ve turnuva

optimizasyon.py · ga()

```
def ga(f, lb, ub, pop=30, iters=60, seed=0,
       pc=0.9, pm=0.15, elit=2, tur=3, alfa=0.5):
    r = np.random.default_rng(seed)
    lb, ub = np.asarray(lb, float), np.asarray(ub, float)
    D, aralik = len(lb), ub - lb
    X = lb + r.random((pop, D)) * aralik
    F = f(X)
    egri = [F.min()]
    n_ev, butce = pop, pop * iters
    while n_ev < butce:
        sira = np.argsort(F)
        def secim():
            aday = r.integers(0, pop, tur)
            return X[aday[np.argmin(F[aday])]]
        n_c = min(pop - elit, butce - n_ev)
        cocuk = []
```

Satır	Basitçe
def ga(...)	Girişler: puan makinesi f, sınırlar, 30 set, %90 çaprazlama, %15 mutasyon, 2 şampiyon, 3'lü turnuva, pay 0,5
r = ...(seed)	Rastgele sayı aracı; aynı seed → aynı sonuç
D, aralik	6 sayı; aralık = üst − alt [2, 0,5, 1, ...]
X = lb + ...	30 rastgele set (30 × 6 tablo), her sayı kendi sınırında
F = f(X)	30 seti 9 uçuşa sok → 30 J
n_ev, butce	Sayaç 30, hak 1800
while ...	Ana döngü = şemadaki geri dönüş oku
sira	Setleri en iyiden kötüye sırala
secim()	Turnuva: rastgele 3 → en küçük J'li
n_c	Yapılacak çocuk: 28 (son nesilde kalan kadar)

Buraya kadar: 30 set üret, puanla, döngüye gir, turnuva aracını hazırla.

ga() kodu adım adım (2/2): çocuk üretimi ve yeni nesil

optimizasyon.py · ga() devam

```
        while len(cocuk) < n_c:
            p1, p2 = secim(), secim()
            if r.random() < pc:
                lo, hi = np.minimum(p1,p2), np.maximum(p1,p2)
                c = r.uniform(lo - alfa*(hi-lo),
                              hi + alfa*(hi-lo))
            else:
                c = p1.copy()
            mut = r.random(D) < pm
            c[mut] += r.normal(0, 0.1 * aralik[mut])
            cocuk.append(np.clip(c, lb, ub))
        Xc = np.array(cocuk); Fc = f(Xc); n_ev += n_c
        kalan = sira[elit:pop-n_c+elit] if n_c < pop-elit else sira[:0]
        X = np.vstack([X[sira[:elit]], X[kalan], Xc])
        F = np.concatenate([F[sira[:elit]], F[kalan], Fc])
        egri.append(F.min())
    i = np.argmin(F)
    return X[i], F[i], np.minimum.accumulate(egri)
```

Satır	Basitçe
p1, p2	2 turnuva → anne, baba
if r.random() < pc	10'da 9 çaprazla
lo, hi, uniform	Her sütunda küçük/büyük, pay ekle, aralıktan rastgele seç
else: p1.copy()	10'da 1 annenin kopyası
mut = ... < pm	Hangi sayılar oynatılacak? (her biri %15)
c[mut] += normal	Seçilene küçük ekleme (aralığın ~%10'u)
np.clip	Sınır kontrolü
Fc = f(Xc)	28 çocuk birlikte 9 uçuşa
X = vstack(...)	Yeni nesil: 2 şampiyon + 28 çocuk
return	En iyi 6 kazanç, en iyi J, eğri

Ders şeması: f(X) = "sistemi analiz et" · while = geri dönüş oku · butce = "DUR".

GA ayarları nereden geliyor?

Ayar	Değer	Kaynak / gerekçe
Çaprazlama yöntemi, alfa	BLX-α, α = 0,5	Eshelman & Schaffer (1993): yöntemi tanıtan makale, en iyi sonuç α = 0,5
pm (mutasyon ihtimali)	0,15 ≈ 1/6	"1 ÷ sayı adedi" kuralı (Mühlenbein 1992; Bäck 1993)
pc (çaprazlama ihtimali)	0,90	Literatürde genelde 0,6–0,95 (Grefenstette 1986: 0,95)
tur (turnuva)	3	Genelde 2–3 (Goldberg & Deb 1991); zayıflara da şans kalır
elit	2	Elitizm: De Jong (1975); en iyiyi asla kaybetme
Mutasyon büyüklüğü	aralığın %10'u	Pratik tercih (kaynak yok): küçük ama etkili
POP × ITER	30 × 60 = 1800	Bütçe kararı (~9 s/koşu); Grefenstette (1986) de POP = 30 önermiş

Hoca sorarsa: "Bu ayarları denediniz mi?" Hayır, duyarlılık analizi yok. Ama GA'nın birbirinden bağımsız 10 koşusunun **hepsi** aynı sonuca ulaştı (J ≈ 0,6592, 10/10) → bulunan çözüm en iyiye çok yakın.

Bildiri "Sınırlılıklar"a: "Algoritma parametreleri literatürdeki yaygın değerlerden seçilmiş, duyarlılık analizi yapılmamıştır."

Karşılaştırma için: Rastgele Arama (taban çizgisi)

Rastgele Arama nedir?

- Gözü kapalı dart: her turda 30 rastgele set dene, en iyisini aklında tut.
- **Hiç öğrenmez:** önceki iyi setlere bakmaz, çocuk üretmez.
- Aynı bütçe: 1800 deneme, 10 koşu.
- Amacı: “GA gerçekten öğreniyor mu?” sorusunu cevaplamak.

GA ne farklı yapıyor?

- İyi setleri **seçiyor** (turnuva).
- İyileri **karıştırıyor** (çaprazlama) ve biraz **değiştiriyor** (mutasyon).
- En iyileri **saklıyor** (elitizm) → J hiç kötüleşmiyor.
- Yani her nesil öncekinden **öğreniyor**.

	GA	Rastgele Arama
En iyi J / ortalama J	0,659 / 0,659	0,709 / 0,732
%1’e ulaşan koşu	10/10 (~18. nesilde)	0/10
10 koşunun dağılımı (std)	$9{,}3\times10^{-5}$ (çok dar)	$1{,}6\times10^{-2}$ (geniş)

İkisi de aynı kural: alg(f, sınırlar, 30, 60, seed) → (en iyi ayar, en iyi J, eğri) · bütçe 1800 deneme.

Koşu › nesil › set: GA kaç nesilde buluyor?

Terim	Ne?	Kaç tane?
Set	6 sayı: Kp1 Ki1 Kd1 Kp2 Ki2 Kd2	—
Nesil	30 set (2 şampiyon + 28 çocuk)	koşuda 64
Koşu	GA'nın baştan sona 1 kez çalışması	toplam 10

İç içe kutular

```
10 koşu  (for s in range(1, 11))
 └─ her koşu = 64 nesil  (while)
     └─ her nesil = 30 set
         └─ her set = 6 sayı

Her koşu yeni 30 rastgele setle
sıfırdan başlar; nesil hep 30 set kalır.
```

Nesil	Deneme	En iyi J (10 koşu ort.)
0 (rastgele)	30	0,886
5	170	0,713
10	310	0,680
~18	**~520**	**0,666 ← %1 yakın**
~36	~1030	0,660 ← %0,1 yakın
64 (son)	1800	0,659

- Yazılım 18'de **durmaz**; bütçe bitene kadar 64 nesil çalışır. 18, eğriye **sonradan** bakılarak ölçüldü.
- İlk 10 nesil büyük iniş, 10–18 iyi bölge, 18'den sonrası ince ayar.

Sonuç: GA en iyinin %1 yakınına ~18 nesilde (≈ 520 deneme) ulaşıyor; kalan ~46 nesil ince ayar.

07

Deneyler ve sonuçlar #src/deneyler.py

10 koşu, 200 yeni test senaryosu, şekiller ve ablasyon çalışması.

deneyler.py: deney yöneticisi

Asıl çalıştırılan dosya: GA'yı ve karşılaştırma için rastgele aramayı 10'ar kez çalıştırır, karşılaştırır, en iyi ayarı 200 yeni testte dener, her şeyi sonuclar/ klasörüne yazar.

Parça	Basitçe ne yapıyor?
MANUEL, N_TEST, TEST_SEED	Elle ayarlı PID [0,5 · 0,05 · 0,05]; son test: 200 senaryo, kâğıt no 12345
KayitliAmac	Not tutan puan makinesi: her soruda "kaç deneme, en iyi J?" → yakınsama eğrisi. seed = 0 burada
test_metrikleri(x)	Son sınav: ayarı hiç görmediği 200 testte uçur, karne çıkar (J, sapma, max-Q ihlali…)
main() a–c	Her yöntem × 10 koşu (tohum s = 1…10), her koşuya temiz makine → tum_kosular.csv
main() d–e	Özet: min/ort/std, %1'e ulaşma, Mann-Whitney p → ozet_egitim.csv
main() f–g	Manuel + en iyi ayarlar 200 testte → test_sonuclari.csv; eğriler → yakinsama.json
if __name__ == "__main__"	python deneyler.py → main(). Import edilince çalışmaz

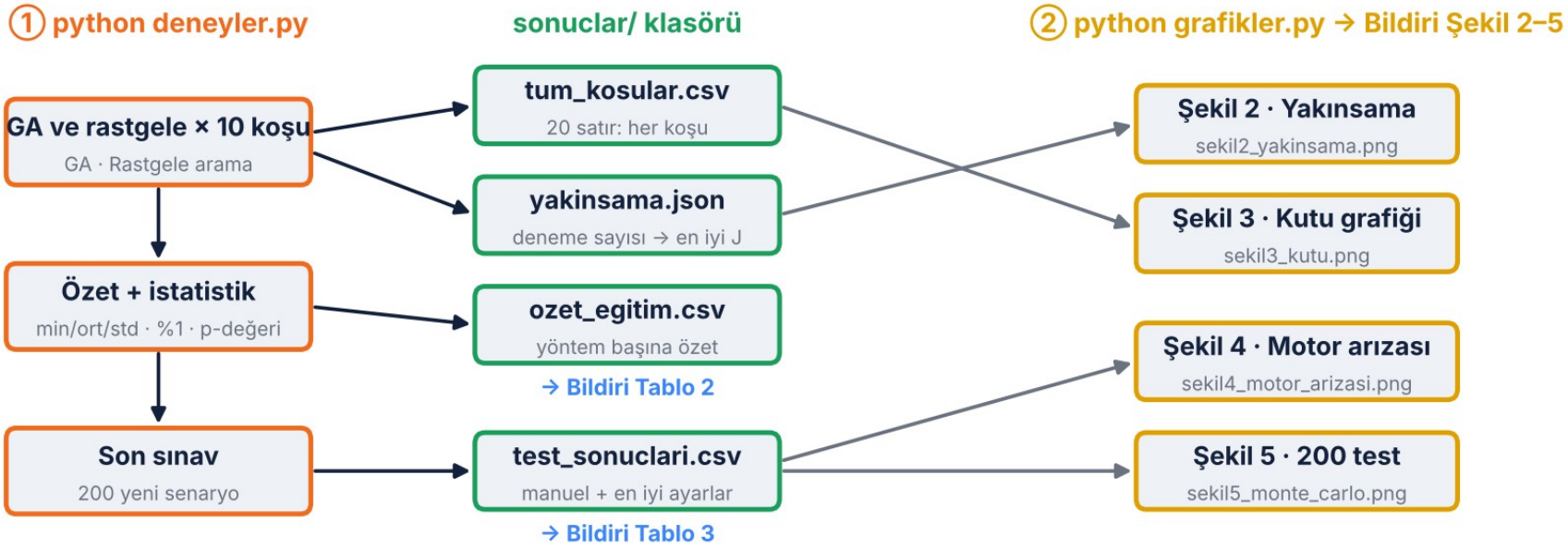
main() özü

```
for ad, alg in ALGORITMALAR:
  for s in range(1, 11):
    f = KayitliAmac()
    x, J, _ = alg(f, LOWER,
        UPPER, 30, 60, seed=s)
    # sonucu tabloya ekle

# özet + Mann-Whitney
# en iyiler 200 testte
# CSV ve JSON kaydet
```

Bir koşu = 1800 deneme × 9 test = 16.200 sahte uçuş ≈ 9 saniye; 20 koşu + test ≈ 4 dakika.

deneyler.py çıktı haritası



tum_kosular.csv: 20 koşunun hepsi	yakinsama.json: deneme → en iyi J	ozet_egitim.csv: Tablo 2	test_sonuclari.csv: Tablo 3

Önce deneyler.py (dosyaları üretir), sonra grafikler.py (dosyaları okuyup çizer).

Çıktı: ozet_egitim.csv → Bildiri Tablo 2

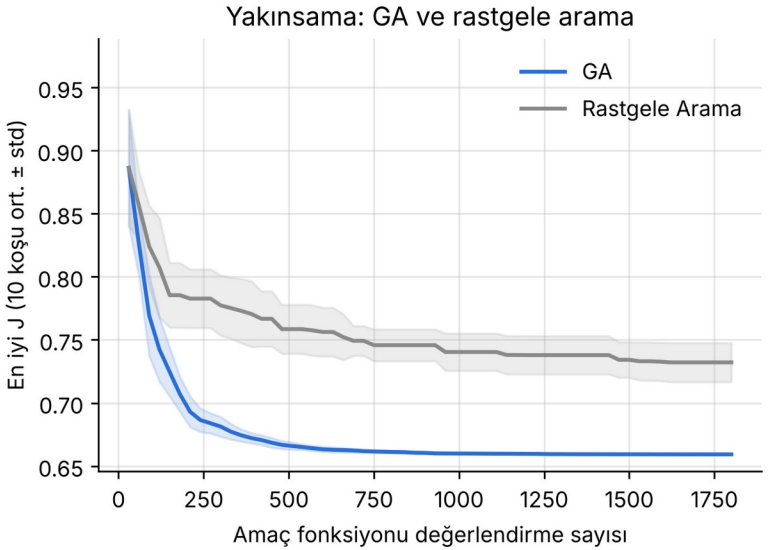
Yöntem	En iyi J	Ortalama J	Std. sapma	%1'e ulaşma	Başarı	Süre/koşu	p (GA ile)
GA	0,65916	0,65924	$9{,}3\times10^{-5}$	519 deneme	10/10	9,5 s	—
Rastgele arama	0,70922	0,73213	$1{,}6\times10^{-2}$	—	0/10	8,9 s	0,0002

Nasıl okunur?

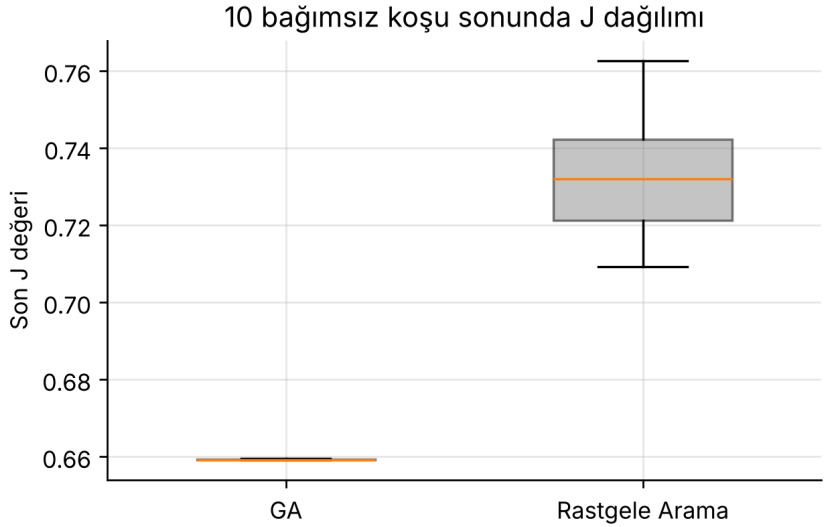
Sütun	Anlamı	Sonuç
En iyi / ortalama J	10 koşunun en iyisi ve ortalaması (küçük = iyi)	GA'nın 10 koşusu aynı noktaya ulaştı; rastgele çok geride
Std. sapma	10 koşu birbirine ne kadar yakın? (istikrar)	GA çok istikrarlı: rastgeleden ~175 kat dar
%1'e ulaşma	En iyi puanın %1 yakınına kaç denemede ulaşıldı (hız)	GA ~18. nesilde (519 deneme); rastgele hiç
p-değeri	Fark şans mı? $p < 0{,}05$ ise gerçek	GA–rastgele farkı gerçek ($p = 0{,}0002$)

Kısaca: GA'nın 10 koşusunun hepsi aynı en iyi çözümü buluyor; rastgele arama hiç ulaşamıyor.

Çıktı: Şekil 2 (yakınsama) ve Şekil 3 (kutu grafiği)



Yakınsama: yatay eksen deneme sayısı, dikey en iyi J. GA (mavi) ilk ~500 denemede hızla iniyor, sonra 0,659'a oturuyor. Rastgele (gri) 0,73'te takılıyor.



Kutu grafiği: 10 koşunun son J dağılımı. GA kutusu 0,659'da çizgi kadar ince (çok istikrarlı); rastgele geniş ve yüksek.

Kaynak: yakinsama.json → Şekil 2 · tum_kosular.csv → Şekil 3 (grafikler.py)

Çıktı: test_sonuclari.csv → Bildiri Tablo 3

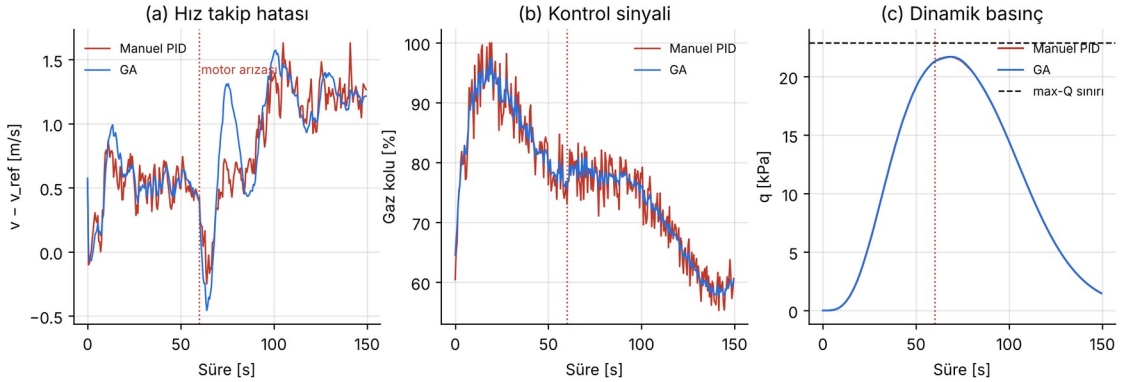
Ayar	J (test)	Sapma IAE	Son hız hatası ort / maks	Max-Q ihlali	En yüksek ivme	Gaz oynaması TV
Manuel PID	0,752	36,2 m	0,19 / 0,73 m/s	0/200	2,39 g	45,6
GA	**0,669**	**38,8 m**	**0,23 / 0,82 m/s**	**0/200**	**2,30 g**	**15,2**
Rastgele arama	0,711	43,2 m	0,17 / 0,72 m/s	0/200	2,35 g	24,4

Nasıl okunur?

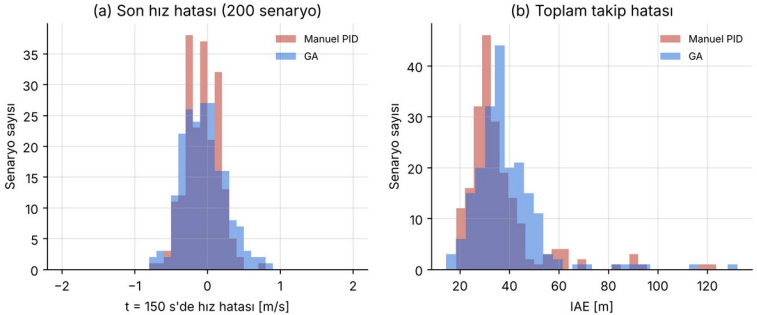
- **J:** optimize ayar elle ayarlı PID'den %11 iyi (0,752 → 0,669).
- **Gaz oynaması:** %67 azaldı (45,6 → 15,2): motorlar çok daha sakin çalışıyor.
- **Bedeli:** sapma %7 arttı (36,2 → 38,8 m), son hız hatası 0,19 → 0,23 m/s; ~1200 m/s'de ihmal edilebilir.
- **Güvenlik:** 200 testin hiçbirinde max-Q sınırı aşılmadı; en yüksek ivme 2,3 g.
- **Ezber yok:** eğitimde 0,659, hiç görmediği testlerde 0,669; neredeyse aynı.

Kısaca: optimizasyon çok küçük bir takip kaybı karşılığında motorları üç kat daha sakin çalıştıran bir ayar buldu.

Çıktı: Şekil 4 (motor arızası) ve Şekil 5 (200 test)



- **Şekil 4:** 60. saniyede bir motor bozuluyor (IFT-8).
- (a) Hata kısa süre düşüyor, PID 2 m/s altında toparlıyor.
- (b) GA'nın gaz kolu (mavi) manuelden (kırmızı) çok daha sakin.
- (c) Basınç sınırın altında.



- **Şekil 5:** 200 testin dağılımı (kırmızı manuel, mavi GA).
- (a) Son hız hatası: ikisi de ±1 m/s içinde.
- (b) Toplam sapma (IAE): GA biraz sağa kaymış (%7 daha fazla). Gaz oynamasındaki büyük kazancın küçük bedeli.

Kaynak: test_sonuclari.csv'deki ayarlar → grafikler.py yeniden uçurup çiziyor.

Ablasyon çalışması: tek faz mı, iki faz mı?

Ablasyon nedir?

- Sistemden **bir parçayı çıkarıp** sonucun ne kadar değiştiğine bakmak.
- Amaç: her parçanın **gerçek katkısını** sayıyla göstermek.
- Burada çıkardığımız parça: **kazanç programlama.** Tek faz = bütün uçuş boyunca aynı Kp, Ki, Kd (3 sayı).
- Aynı yöntem (GA, 5 koşu), aynı senaryolar, aynı J; değişen tek şey faz sayısı.

	Tek faz	İki faz	Fark
Parametre sayısı	3	6	+3
Bulunan Kp	0,203	0,230 → 0,155	—
J (eğitim, 9 senaryo)	0,6611	0,6592	%0,3 daha iyi
J (test, 200 senaryo)	0,6707	0,6694	%0,2 daha iyi
Hız sapması IAE	39,0	38,8	%0,6 daha iyi
Gaz oynaması TV	15,4	15,2	%1,3 daha iyi
Max-Q ihlali	0/200	0/200	aynı

Neden fark bu kadar küçük?

- **İleri besleme** hava incelmesini, yerçekimini ve direnci zaten baştan karşılıyor; PID sadece küçük düzeltme yapıyor.
- **Max-Q sınırı zorlanmıyor** (~21,9 kPa, sınır ~23 kPa); Faz-1'in "yumuşak ol" görevi kendiliğinden sağlanıyor.
- Kamyon örneğinde fark büyüktü çünkü orada ileri besleme yoktu.

Sonuç: iki faz yalnızca küçük bir ince ayar (%0,2–0,3). Değişen dinamiği büyük ölçüde ileri besleme karşılıyor. Faz geçişini değil, kontrol yapısını ileri besleme taşıyor.

08

Özet ve devam eden çalışmalar

Ne yaptık, ne bulduk ve bundan sonra ne yapacağız?

Genel özet

1 — Veri

Kaggle: IFT-6/7/8 booster, ilk 150 s

2 — Temizleme

Tekrarlar, km/h→m/s, polinomla yumuşatma, γ, max-Q

3 — Sahte roket

Fizik modeli: itki, yerçekimi, direnç, kütle, motor gecikmesi

4 — Kontrolcü

PID + ileri besleme + kazanç programlama (2 faz)

5 — Senaryolar

9 zor sürüş: ağırlık, direnç, motor gücü, arıza, gürültü

6 — Puan J

Sapma + son hız + max-Q cezası + yakıt + gaz oynaması

7 — Optimizasyon

GA 6 kazancı arıyor; 10 koşu × 64 nesil

Sonuçlar

Soru	Cevap
GA ne buldu?	10 koşunun hepsi aynı noktaya ulaştı (J ≈ 0,659); rastgele arama 0,709'da kaldı
Ne kadar hızlı?	En iyinin %1 yakınına ~18. nesilde (≈ 519 deneme) ulaştı; kalan nesiller ince ayar
Ne kadar güvenilir?	10/10 koşu başarılı; std $9{,}3 \cdot 10^{-5}$; rastgele aramadan farkı anlamlı (p = 0,0002)
200 yeni senaryoda test	Elle ayara göre J 0,752 → 0,669; gaz oynaması −%67; max-Q ihlali 0/200
Ablasyon: iki faz ne kazandırdı?	Tek faza göre yalnızca %0,2–0,3; asıl işi ileri besleme yapıyor

Kısaca: gerçek uçuş verisinden kurulan sahte rokette, ileri beslemeli ve iki fazlı PID'in 6 kazancını Genetik Algoritma otomatik ayarladı: elle ayara göre %11 daha iyi, motorlar 3 kat daha sakin.

DEVAM EDEN ÇALIŞMALAR

Sırada ne var? Karınca Kolonisi (ACO) ile karşılaştırma

1 Literatür taraması

- PID kazançlarının metasezgisel yöntemlerle ayarlanması üzerine çalışmalar
- Roket ve uçuş kontrolünde GA ve ACO uygulamaları
- Önerilen parametre değerleri ve karşılaştırma ölçütleri

2 ACO mantığını öğrenmek

- Karıncalar, feromon ve buharlaşma fikri
- Sürekli uzay için ACO_R: en iyi çözüm arşivi, rehber seçimi
- Arama alanının giderek daralması; kodun adım adım okunması

3 GA ve ACO karşılaştırması

- Aynı koşullar: 1800 deneme, aynı 9 senaryo, 10 koşu
- Ölçütler: en iyi / ortalama J, istikrar, %1'e ulaşma hızı
- Mann-Whitney testi ve 200 yeni test senaryosunda doğrulama

Hedef: GA'yı ikinci bir yöntemle adil koşullarda karşılaştırıp sonuçları bildiriye eklemek.